



BASIC · INTERMEDIATE · ADVANCED

Python Practical Handbook

পাইথন প্র্যাকটিক্যাল হ্যান্ডবুক — ইংরেজি ও বাংলা

From zero to production-ready scripts through real problem-solving: bill calculators, login limiters, sales reports, log analyzers, inventory systems, and a full API + database capstone. Every concept: English first, then বাংলা. No filler — labs, mistakes, exercises with answers.

8 Chapters

8 Real-World Labs

60+ Runnable Examples

Exercises + Answers

English + বাংলা

BASIC · INTERMEDIATE · ADVANCED

Python Practical Handbook

পাইথন প্র্যাকটিক্যাল হ্যান্ডবুক — ইংরেজি ও বাংলা

From zero to production-ready scripts through real problem-solving: bill calculators, login limiters, sales reports, log analyzers, inventory systems, and a full API + database capstone. Every concept: English first, then বাংলা. No filler — labs, mistakes, exercises with answers.

8

CHAPTERS

8

REAL LABS

60+

CODE EXAMPLES

2

LANGUAGES
EN+বাং

Contents • সূচিপত্র

Module 00 — Start Here: Setup & How to Learn	1
0.1 Who This Book Is For	1
0.2 What Python Is	1
0.3 Installing Python	1
0.4 pip, Virtual Environments	2
0.5 How to Practice	3
Module 01 — Python Basics: Variables, Types & Strings	4
1.1 Variables	4
1.2 Numbers	4
1.3 Strings	5
1.4 Input, Output	5
1.5 Lab: Electricity Bill	6
1.6 Common Mistakes	7
1.7 Practice Exercises	7
1.8 Summary	7
Module 02 — Control Flow & Functions	9
2.1 Decisions	9
2.2 Loops	9
2.3 break, continue	10
2.4 Functions	11
2.5 args, kwargs	11
2.6 Lab: Login Attempt Limiter	11
2.7 Common Mistakes	12
2.8 Practice Exercises	13
Module 03 — Data Structures That Run Real Systems	14
3.1 Lists	14
3.2 Tuples	14
3.3 Dictionaries	15
3.4 Sets	15
3.5 Comprehensions	16
3.6 Lab: Top-Selling Products	16
3.7 Common Mistakes	17
3.8 Practice Exercises	17
Module 04 — Files, Errors, JSON & CSV	19
4.1 Reading	19
4.2 Errors	19
4.3 Raising Your Own Errors	20
4.4 JSON	20
4.5 CSV	21
4.6 Lab: Server Log Analyzer	21
4.7 Common Mistakes	22
4.8 Practice Exercises	23
Module 05 — Object-Oriented Python & Project Structure	24
5.1 Classes	24
5.2 repr	24

5.3 Inheritance	25
5.4 dataclass	25
5.5 Modules, Packages	26
5.6 Lab: Inventory Manager	26
5.7 Common Mistakes	28
5.8 Practice Exercises	28
Module 06 — Advanced Python: Elegant & Efficient Code	30
6.1 Generators	30
6.2 Decorators	30
6.3 Context Managers	31
6.4 Type Hints	32
6.5 functools	32
6.6 Lab: Retry Decorator	33
6.7 Common Mistakes	34
6.8 Practice Exercises	34
Module 07 — Production Python: Real Tools for Real Jobs	36
7.1 Logging	36
7.2 Command-Line Tools	36
7.3 Calling Web APIs	37
7.4 SQLite	37
7.5 Testing with pytest	38
7.6 Concurrency	39
7.7 Capstone Lab	39
7.8 Production Checklist	40
Appendix — Cheat Sheet, Error Decoder & Next Steps	42
A.1 One-Page Cheat Sheet	42
A.2 Error Message Decoder	43
A.3 Ten Real Projects	43
A.4 Where to Go Next	44

Start Here: Setup & How to Learn

শুরু এখানেই: সেটআপ ও শেখার নিয়ম

IN THIS CHAPTER · এই অধ্যায়ে

Who this book is for · what Python is · install & first program · pip and virtual environments · the 20-minute practice rule.

0.1 Who This Book Is For & How to Read It

This book takes you from **zero Python** to writing **production-style scripts** that solve real problems: bill calculators, login limiters, sales reports, log analyzers, inventory systems, API fetchers with databases and tests. No filler theory — every chapter ends in working code you could actually ship.

Each chapter follows the same skeleton: **concept** → **mechanism** → **real example** → **hands-on lab** → **common mistakes** → **practice exercises with answers** → **summary**. Every section is English first, then a বাংলা panel that restates the core idea in short, natural Bangla (a rewrite, not a word-for-word translation).

★ KEY POINT

Do not just read. **Type every example yourself**, break it on purpose, and fix it. Typing → error → fix is how programming actually enters your brain.

বাংলা

এই বইটি আপনাকে শূন্য থেকে শুরু করে বাস্তব সমস্যা সমাধানের Python কোড লেখা শেখাবে — বিদ্যুৎ বিল ক্যালকুলেটর, লগইন লিমিটার, সেলস রিপোর্ট, লগ অ্যানালাইজার, API + ডাটাবেস প্রজেক্ট পর্যন্ত। অপ্রয়োজনীয় থিংগরি নেই। প্রতিটি সেকশন আগে ইংরেজি, তারপর সংক্ষিপ্ত বাংলা। সবচেয়ে গুরুত্বপূর্ণ নিয়ম: **শুধু পড়বেন না, প্রতিটি কোড নিজে টাইপ করুন**, ইচ্ছা করে ভাঙুন, তারপর ঠিক করুন — এভাবেই শেখা হয়।

0.2 What Python Is & Where It Is Used

Plain-language first: Python is a language for giving instructions to a computer, designed to read almost like English. You write instructions in a text file; a program called the **interpreter** reads that file top-to-bottom and executes each line.

Think of a recipe: you (the programmer) write the recipe, and the interpreter is the cook who follows it exactly — including your mistakes. Python is used for web backends (Instagram, YouTube parts), data analysis and AI (pandas, PyTorch), automation/DevOps scripts, and quick tools that glue systems together. It is the most common "second language" for testers, sysadmins, analysts and engineers because a useful script is often only 30 lines.

Language type	How it runs	Example
Compiled	Whole file translated to machine code first, then run	C, C++, Go
Interpreted	Read and executed line by line by an interpreter	Python , JavaScript

বাংলা

Python হলো কম্পিউটারকে নির্দেশ দেওয়ার একটি ভাষা, যা প্রায় ইংরেজির মতো পড়া যায়। আপনি একটি টেক্সট ফাইলে নির্দেশ লেখেন, আর **ইন্টারপ্রেটার** নামের প্রোগ্রামটি উপর থেকে নিচে লাইন ধরে ধরে তা চালায় — অনেকটা রেসিপি অনুযায়ী রান্না করা বাবুর্চির মতো। ওয়েব ব্যাকএন্ড, ডেটা অ্যানালাইসিস, AI, অটোমেশন — সবখানেই Python ব্যবহৃত হয়। মাত্র ৩০ লাইনের স্ক্রিপ্টও বাস্তব কাজে লাগে।

0.3 Installing Python & Running Your First Program

Install Python **3.10 or newer** from python.org/downloads. On Windows, tick **"Add Python to PATH"** during install — skipping this causes the single most common beginner error (`'python' is not recognized`). Verify from a terminal

(Command Prompt / PowerShell / macOS Terminal / Linux shell):

TERMINAL

```
python --version      # Windows
python3 --version     # macOS / Linux usually needs python3
```

Create a file named `hello.py` in any folder using any editor (**VS Code** recommended — free, with the official Python extension):

PYTHON

```
name = "Rahim"
print("Hello,", name)
print("2 + 3 =", 2 + 3)
```

Run it from the terminal, from the folder where the file lives:

TERMINAL

```
python hello.py
```

OUTPUT

```
Hello, Rahim
2 + 3 = 5
```

You can also type `python` alone to enter the **REPL** (Read-Evaluate-Print Loop) — an interactive prompt where each line runs immediately. Great for quick experiments; exit with `exit()`.

⚠ WARNING

Never name your own file the same as a standard module — `random.py`, `json.py`, `csv.py`, `test.py`. Python will import **your** file instead of the real library and you will get baffling errors like `AttributeError: module 'random' has no attribute 'randint'`.

বাংলা

python.org থেকে Python 3.10+ ইনস্টল করুন; Windows-এ ইনস্টলের সময় **"Add Python to PATH"** টিক দিতে ভুলবেন না। এরপর টার্মিনালে `python --version` দিয়ে যাচাই করুন। VS Code-এ `hello.py` ফাইল বানিয়ে `python hello.py` কমান্ডে চালান। শুধু `python` লিখলে **REPL** খোলে — যেখানে প্রতিটি লাইন সাথে সাথে চলে; পরীক্ষা-নিরীক্ষার জন্য দারুণ। সতর্কতা: নিজের ফাইলের নাম কখনো `random.py`, `json.py` — এমন লাইব্রেরির নামে রাখবেন না।

0.4 pip, Virtual Environments & a Clean Project Folder

pip is Python's package installer — it downloads libraries other people wrote (from PyPI, the Python Package Index) so you don't rebuild everything yourself. A **virtual environment (venv)** is a private, per-project copy of Python + its packages, so Project A needing `requests 2.31` never breaks Project B needing an older one. Professional rule: **one project, one venv. Always.**

TERMINAL

```
mkdir bill-app && cd bill-app
python -m venv .venv          # create the environment

# activate it:
.venv\Scripts\activate      # Windows
source .venv/bin/activate    # macOS / Linux

pip install requests         # installs only inside this venv
```

```
pip freeze > requirements.txt      # record exact versions for teammates
pip install -r requirements.txt    # teammate restores the same setup
```

Your prompt shows `(.venv)` when active. A healthy small-project layout:

OUTPUT

```
bill-app/
├── .venv/          # never commit this to git
├── main.py         # entry point
├── helpers.py      # your reusable functions
└── requirements.txt # pinned dependencies
```

বাংলা

pip দিয়ে অন্যদের বানানো লাইব্রেরি ইনস্টল করা হয়। **venv** হলো প্রতিটি প্রজেক্টের নিজস্ব আলাদা Python পরিবেশ — এক প্রজেক্টের প্যাকেজ আরেকটাকে ভাঙতে পারে না। পেশাদার নিয়ম: **এক প্রজেক্ট, এক venv**। `python -m venv .venv` দিয়ে বানান, activate করুন, তারপর `pip install`। `pip freeze > requirements.txt` দিয়ে ভার্সন লিখে রাখুন — সহকর্মী একই কমান্ডে হুবহু একই সেটআপ পাবে। `.venv` ফোল্ডার কখনো git-এ দেবেন না।

0.5 How to Practice: the 20-Minute Rule

- **Type, don't copy-paste.** Muscle memory matters.
- **Predict before you run.** Say the output out loud, then run. Wrong guess = the exact spot you're about to learn something.
- **The 20-minute rule:** when stuck, struggle alone for 20 minutes (read the error bottom-up, print variables), *then* search. Struggle builds the skill; searching too early builds dependence.
- **Finish every lab and exercise.** They are the actual course; the prose is just the setup.
- Read error messages **from the last line upward** — the last line names the error, the lines above show where.

বাংলা

অনুশীলনের নিয়ম: কোড **টাইপ করুন**, কপি-পেস্ট নয়। রান করার আগে আউটপুট **অনুমান করুন**। আটকে গেলে **২০ মিনিট** নিজে চেষ্টা করুন — এরর মেসেজ নিচ থেকে উপরে পড়ুন, ভেরিয়েবল print করে দেখুন — তারপর সার্চ করুন। প্রতিটি ল্যাব ও এক্সারসাইজ শেষ করুন; ওগুলোই আসল কোর্স।

Python Basics: Variables, Types & Strings

পাইথন বেসিক: ভেরিয়েবল, টাইপ ও স্ট্রিং

IN THIS CHAPTER · এই অধ্যায়ে

Core types · math operators that matter (`//`, `%`) · f-strings and the 7-method text kit · input/output · Lab: slab-priced electricity bill.

1.1 Variables & Data Types

Plain-language first: a variable is a labelled box that stores a value so you can use it later by name.

Python creates the box the moment you assign with `=`. You never declare a type — Python infers it from the value. The four core types you'll use constantly:

PYTHON

```
customer = "Ayesha Traders"    # str — text, in quotes
units    = 245                  # int  — whole number
rate     = 7.85                 # float — decimal number
is_paid  = False                # bool — True or False only

print(type(units))              # <class 'int'>
```

Naming rules that matter in real code: use `snake_case` (`unit_price`, not `UnitPrice`), make names *say what they hold* (`retry_count`, never `x` or `data2`), and never start with a digit. Names are case-sensitive: `Rate` and `rate` are different boxes.

★ KEY POINT

A variable in Python is a **name pointing at a value**, not the value itself. `b = a` makes both names point at the *same* value — this matters enormously for lists later (3.1).

বাংলা

ভেরিয়েবল হলো নাম লেখা একটি বাক্স, যাতে মান রাখা হয়। `=` দিয়ে অ্যাসাইন করলেই বাক্স তৈরি হয়; টাইপ বলে দিতে হয় না। মূল চার টাইপ: `str` (টেক্সট), `int` (পূর্ণসংখ্যা), `float` (দশমিক), `bool` (True/False)। নাম দিন `snake_case`-এ এবং অর্থবহভাবে — `x` নয়, `unit_price`। মনে রাখুন: ভেরিয়েবল মান নয়, মানের দিকে **নির্দেশক**; `b = a` মানে দুটি নাম একই মানের দিকে তাকিয়ে আছে।

1.2 Numbers & Math

The operators, including three that beginners meet late but professionals use daily:

PYTHON

```
print(17 + 5, 17 - 5, 17 * 5)    # 22 12 85
print(17 / 5)                    # 3.4    true division — ALWAYS returns float
print(17 // 5)                   # 3      floor division — throws away the remainder
print(17 % 5)                    # 2      modulo — the remainder itself
print(2 ** 10)                   # 1024   power
```

Real uses: `%` checks even/odd (`n % 2 == 0`) and "every Nth item"; `//` converts 130 minutes into 2 hours (`130 // 60`) with `%` giving the leftover 10 minutes. Shorthand updates: `total += price` means `total = total + price` (also `-=`, `*=`, `/=`).

⚠ WARNING

Floats are approximate: `0.1 + 0.2` prints `0.30000000000000004`. Never compare floats with `==`, and never store money as float in serious systems — use `int` paisa/cents, or the `decimal` module. `round(x, 2)` is fine for *display*.

বাংলা

সাধারণ অপারেটরের সাথে তিনটি বিশেষ জিনিস শিখুন: `/` সবসময় দশমিক দেয়, `//` ভাগফলের পূর্ণ অংশ দেয়, `%` ভাগশেষ দেয়। বাস্তব ব্যবহার: `n % 2 == 0` মানে জোড় সংখ্যা; `130 // 60` = ২ ঘণ্টা, `130 % 60` = ১০ মিনিট। সতর্কতা: float আনুমানিক — `0.1 + 0.2` ঠিক `0.3` হয় না, তাই টাকার হিসাব float-এ রাখবেন না (পয়সায় int রাখুন), আর float কখনো `==` দিয়ে তুলনা করবেন না।

1.3 Strings & f-strings

A string is text in quotes (`'...'` or `"..."` — identical). The tool you will use in almost every line of real output is the **f-string**: put `f` before the quote and drop variables/expressions inside `{}`:

PYTHON

```
name, due = "Karim Store", 12459.5
print(f"Dear {name}, your due is {due:,.2f} BDT") # Dear Karim Store, your due is 12,459.50 BDT
print(f"{'ID':<6}{'Price':>10}") # column alignment
print(f"{name.upper()} has {len(name)} characters") # expressions work inside {}
```

Format specs after the `:` — `,` thousands separator, `.2f` two decimals, `<6 / >10` left/right align in a width. Essential string methods (strings are **immutable**: methods return a *new* string, they never modify in place):

PYTHON

```
raw = " ORD-1043/Dhaka "
clean = raw.strip() # "ORD-1043/Dhaka" — trim whitespace
print(clean.lower()) # ord-1043/dhaka
print(clean.replace("/", " | ")) # ORD-1043 | Dhaka
print(clean.split("/")) # ['ORD-1043', 'Dhaka']
print(clean.startswith("ORD")) # True
print("-".join(["2026", "08", "19"])) # 2026-08-19
print(clean[0:3], clean[-5:]) # ORD Dhaka — slicing [start:stop]
```

`strip` → `lower/upper` → `split` → `join` → `replace` → `startswith/endswith` → `in` — this seven-method kit cleans 90% of the messy text you'll meet in real files.

বাংলা

স্ট্রিং মানে উদ্ধৃতির ভেতরে টেক্সট। বাস্তব কোডের সবচেয়ে দরকারি টুল **f-string**: `f"..."` লিখে `{}`-এর ভেতরে ভেরিয়েবল বসান; `:` এর পরে ফরম্যাট — `.2f` মানে কমাসহ দুই দশমিক। স্ট্রিং **অপরিবর্তনীয়** — মেথড নতুন স্ট্রিং ফেরত দেয়, আসলটা বদলায় না। সাতটি মেথড মুখস্থ করুন: `strip`, `lower`, `split`, `join`, `replace`, `startswith`, `in` — বাস্তবের নোংরা ডেটা পরিষ্কারের ৯০% কাজ এতেই হয়।

1.4 Input, Output & Type Conversion

`input()` shows a prompt and returns whatever the user typed — **always as a string**, even if they typed a number. Convert explicitly:

PYTHON

```
units = int(input("Units used this month: ")) # "245" -> 245
rate = float(input("Rate per unit: ")) # "7.85" -> 7.85
print(f"Bill: {units * rate:.2f} BDT")
```

Conversion functions: `int("42")`, `float("3.5")`, `str(99)`, `int(3.9)` → `3` (truncates, doesn't round — use `round(3.9)` for `4`). Invalid input like `int("abc")` crashes with `ValueError` — chapter 4 teaches `try/except` to handle it gracefully.

PLAIN ENGLISH

`input()` returning a string is the #1 first-week bug: `input() + 5` crashes, and `"7" * 3` gives `"777"` not `21`. When math misbehaves, `print(type(x))` first.

বাংলা

`input()` ব্যবহারকারীর লেখা ফেরত দেয় — **সবসময় স্ট্রিং হিসেবে**, সংখ্যা টাইপ করলেও। তাই অঙ্কের আগে `int()` বা `float()` দিয়ে রূপান্তর করুন। `int(3.9)` হয় `3` (রাউন্ড নয়, কেটে ফেলে); রাউন্ড চাইলে `round()`। ভুল ইনপুটে (`int("abc")`) প্রোগ্রাম ক্র্যাশ করে `ValueError` দিয়ে — চতুর্থ অধ্যায়ে এটা সামলানো শিখবেন। সংখ্যা অদ্ভুত আচরণ করলে আগে `print(type(x))` করুন।

1.5 Lab: Electricity Bill Calculator (Slab Pricing)

Real scenario: Bangladesh's DESCO/DPDC-style tariffs charge different rates per *slab* — the first 75 units at one rate, the next slabs at higher rates. This exact pattern (tiered pricing) also runs mobile data packs, income tax brackets and cloud billing. Build it with only chapter-1 tools:

PYTHON

```
# bill.py - tiered electricity bill (illustrative rates)
units = int(input("Units consumed: "))

slab1 = min(units, 75)                # first 75 units
slab2 = min(max(units - 75, 0), 125)  # next 125 units (76-200)
slab3 = max(units - 200, 0)           # everything above 200

energy = slab1 * 5.26 + slab2 * 7.20 + slab3 * 9.94
demand = 42.00                        # fixed demand charge
vat = (energy + demand) * 0.05        # 5% VAT
total = energy + demand + vat

print(f"\n{'Slab':<14}{'Units':>7}{'Amount':>12}")
print(f"{'0-75':<14}{slab1:>7}{slab1 * 5.26:>12.2f}")
print(f"{'76-200':<14}{slab2:>7}{slab2 * 7.20:>12.2f}")
print(f"{'200+':<14}{slab3:>7}{slab3 * 9.94:>12.2f}")
print(f"{'Demand charge':<21}{demand:>12.2f}")
print(f"{'VAT 5%':<21}{vat:>12.2f}")
print(f"{'TOTAL':<21}{total:>12.2f} BDT")
```

OUTPUT

```
Units consumed: 245

Slab      Units      Amount
0-75      75          394.50
76-200    125          900.00
200+      45           447.30
Demand charge          42.00
VAT 5%              89.19
TOTAL              1872.99 BDT
```

The `min`/`max` trick replaces a pile of `if` statements: `min(units, 75)` caps a slab at its size; `max(units - 200, 0)` floors it at zero so small consumers don't get negative units. Study those two lines until they're obvious — the pattern reappears in every tiered-pricing system you will ever build.

বাংলা

বাস্তব দৃশ্য: বিদ্যুৎ বিল স্ল্যাবভিত্তিক — প্রথম ৭৫ ইউনিট এক রেটে, পরের ইউনিটগুলো বেশি রেটে। একই প্যাটার্ন মোবাইল ডেটা প্যাক, আয়কর ও ক্লাউড বিলিং-এ চলে। কৌশলটি লক্ষ করুন: `min(units, 75)` স্ল্যাবকে তার সীমায় আটকায়, `max(units - 200, 0)` ঋণাত্মক হওয়া ঠেকায় — অনেক `if` ছাড়াই টার্নার্ড হিসাব হয়ে যায়। এই দুই লাইন সম্পূর্ণ বুঝে তবেই সামনে যান।

1.6 Common Mistakes

Mistake	Symptom	Fix
Doing math on <code>input()</code> directly	<code>TypeError: can only concatenate str</code>	Wrap in <code>int()</code> / <code>float()</code>
<code>=</code> instead of <code>==</code> in a comparison	<code>SyntaxError</code> inside <code>if</code>	<code>=</code> assigns, <code>==</code> compares
Comparing floats with <code>==</code>	<code>0.1 + 0.2 == 0.3</code> is <code>False</code>	<code>abs(a - b) < 1e-9</code> or use ints
Expecting <code>name.upper()</code> to change <code>name</code>	Value looks unchanged	Strings are immutable: <code>name = name.upper()</code>
Shadowing built-ins: <code>sum = 10</code> , <code>list = [...]</code>	<code>TypeError: 'int' object is not callable</code> later	Never name variables <code>sum</code> , <code>list</code> , <code>str</code> , <code>input</code>

বাংলা

সবচেয়ে ঘন ঘন ভুল: `input()`-এর ফলে সরাসরি অঙ্ক করা (আগে `int()` করুন), তুলনায় `==` এর জায়গায় `=` লেখা, float-কে `==` দিয়ে মেলানো, `name.upper()` লিখে ভাবা আসল স্ট্রিং বদলে গেছে (আবার অ্যাসাইন করতে হয়), আর `sum`, `list`-এর মতো বিল্ট-ইন নাম ভেরিয়েবলে ব্যবহার করা — পরে রহস্যময় এরর দেয়।

1.7 Practice Exercises (with Answers)

- Split the bill:** ask for a restaurant total and number of people; print each person's share with 2 decimals, including a 10% service charge.
- Time converter:** given total seconds (e.g. `9375`), print `2h 36m 15s` using only `//` and `%`.
- Username generator:** from `full_name = "Nusrat Jahan Prova"`, produce `nusrat.prova` (first word + last word, lowercase, dot-joined) using `split`, `lower`, and indexing.

PYTHON

```
# --- Answers ---
# 1
total = float(input("Total: ")); people = int(input("People: "))
print(f"Per head: {total * 1.10 / people:.2f}")

# 2
s = 9375
print(f"{s // 3600}h {s % 3600 // 60}m {s % 60}s") # 2h 36m 15s

# 3
full_name = "Nusrat Jahan Prova"
parts = full_name.lower().split()
print(f"{parts[0]}.{parts[-1]}") # nusrat.prova
```

বাংলা

তিনটি অনুশীলন: (১) রেস্টুরেন্ট বিল ভাগ — ১০% সার্ভিস চার্জসহ মাথাপিছু হিসাব; (২) সেকেন্ডকে `2h 36m 15s`-এ রূপান্তর, শুধু `//` ও `%` দিয়ে; (৩) পুরো নাম থেকে `nusrat.prova` স্টাইলের ইউজারনেম বানানো `split`, `lower` ও ইনডেক্সিং দিয়ে। উত্তর উপরে দেওয়া আছে — আগে নিজে চেষ্টা করুন, তারপর মেলান।

1.8 Summary

Variables are names pointing at values; core types are `str`, `int`, `float`, `bool`. `/` gives floats, `//` and `%` split quantities into whole parts and remainders. f-strings (`f"{x:,.2f}"`) are your output workhorse; the seven string methods clean real-world text. `input()` always returns a string — convert before math. `min` / `max` clamp values elegantly in tiered calculations.

সারসংক্ষেপ: চার মূল টাইপ; `/`, `//`, `%`-এর পার্থক্য; আউটপুটের জন্য f-string; টেক্সট পরিষ্কারে সাত স্ট্রিং মেথড; `input()` সবসময় স্ট্রিং দেয়; আর স্ল্যাব-হিসাবে `min` / `max`-এর কৌশল।

Control Flow & Functions

কন্ট্রোল ফ্লো ও ফাংশন

IN THIS CHAPTER · এই অধ্যায়ে

if/elif and truthiness · for/while · break/continue and the search pattern · production-grade functions · *args/**kwargs & scope · Lab: login attempt limiter with lockout.

2.1 Decisions: if / elif / else

Plain-language first: `if` lets your program choose a path based on a condition, exactly like "IF the balance is enough, approve; OTHERWISE reject."

PYTHON

```
amount, balance = 5000, 3200

if amount <= balance:
    print("Approved")
elif amount <= balance + 2000:      # overdraft window
    print("Approved with overdraft fee")
else:
    print("Declined: insufficient funds")
```

Python defines the block by **indentation** (4 spaces) — there are no braces. Conditions use `==`, `!=`, `<`, `>`, `<=`, `>=`, combined with `and`, `or`, `not`. Two professional idioms:

PYTHON

```
if 18 <= age < 60: ...                # chained comparison — one range check
if status in ("paid", "partial"): ...  # membership beats a chain of or's
```

Truthiness: empty things are `False` in a condition — `""`, `0`, `[]`, `{}`, `None`. So `if items:` means "if the list is not empty", and `if not name:` catches blank input.

⚠ WARNING

Check `None` with `is`, never `==`: `if result is None:`. And `if x == 5 or 6:` does **not** mean "5 or 6" — `6` is always truthy, so the condition is always true. Write `if x in (5, 6):`.

বাংলা

`if` দিয়ে প্রোগ্রাম শর্ত অনুযায়ী পথ বেছে নেয়; ব্লক চেনা হয় **ইনডেন্টেশন** (৪ স্পেস) দিয়ে — ব্রেস নেই। শর্ত জোড়া লাগান `and`, `or`, `not` দিয়ে। দুটি পেশাদার কৌশল: রেঞ্জ চেকে `18 <= age < 60`, আর একাধিক মানের বেলায় `status in ("paid", "partial")`। খালি জিনিস (`""`, `0`, `[]`, `None`) শর্তে `False` — তাই `if items:` মানে "লিস্ট খালি নয়"। `None` যাচাই করুন `is None` দিয়ে, `==` নয়।

2.2 Loops: for & while

A **for loop** repeats once per item in a collection — use it when you know *what* you're iterating over. A **while loop** repeats as long as a condition holds — use it when you don't know how many rounds ("until the user quits", "until the API responds").

PYTHON

```
orders = [1250, 890, 2100, 450]
```

```
total = 0
for amount in orders:
    total += amount
print(f"Day total: {total}")           # 4690

for i, amount in enumerate(orders, start=1):
    print(f"Order #{i}: {amount}")     # numbered rows — never for i in range(len(...))

for day in range(1, 8):
    print(f"Day {day}")                # 1,2,...,7 (stop is excluded)
```

PYTHON

```
balance = 10000
month = 0
while balance > 0:
    balance -= 1500
    month += 1
print(f"Money lasts {month} months")   # 7
```

`enumerate()` gives index + item together; `range(start, stop)` excludes `stop`; `zip(a, b)` walks two lists in parallel. Every `while` needs something inside that moves it toward stopping, or it runs forever (Ctrl+C to kill).

বাংলা

`for` লুপ চলে সংগ্রহের প্রতিটি আইটেমের জন্য একবার — কী নিয়ে ঘুরছেন তা জানা থাকলে এটি। `while` চলে শর্ত সত্য থাকা পর্যন্ত — কত বার লাগবে অজানা থাকলে এটি। নম্বরসহ আইটেম চাইলে `enumerate()` ব্যবহার করুন (`range(len(...))` কখনো নয়)। `range(1, 8)` মানে ১ থেকে ৭ — শেষ সংখ্যা বাদ। প্রতিটি `while`-এর ভেতরে এমন কিছু থাকতেই হবে যা শর্তকে মিথ্যার দিকে নেয়, নইলে অনন্ত লুপ।

2.3 break, continue & the Search Pattern

`break` exits the loop immediately; `continue` skips to the next round. Together they express real filtering and searching:

PYTHON

```
transactions = [120, -50, 300, 999999, 80]

for t in transactions:
    if t < 0:
        continue           # skip refunds
    if t > 100000:
        print("Fraud alert — stopping batch"); break
    print(f"Processing {t}")
```

The classic **search pattern** — find the first match, react if nothing matched — uses the little-known `else` on a loop (runs only if the loop finished *without* `break`):

PYTHON

```
for user in users:
    if user["email"] == target:
        print("Found:", user["name"]); break
else:
    print("No such user")           # only when the search failed
```

বাংলা

`break` লুপ তৎক্ষণাৎ শেষ করে; `continue` বর্তমান রাউন্ড বাদ দিয়ে পরেরটিতে যায় — ফিল্টারিং ও সার্চের মূল হাতিয়ার। বিশেষ কৌশল: লুপের সাথে `else` ব্লক চলে কেবল তখনই যখন লুপ `break` ছাড়া স্বাভাবিকভাবে শেষ হয় — অর্থাৎ "খুঁজে পাওয়া যায়নি" কেসটা লেখার সবচেয়ে পরিচ্ছন্ন জায়গা।

2.4 Functions: Reusable Blocks with a Contract

Plain-language first: a function is a named, reusable block — ingredients go in (parameters), a result comes out (`return`).

PYTHON

```
def slab_bill(units, vat_rate=0.05):          # vat_rate has a default
    """Return total bill for consumed units (tiered tariff)."""
    energy = (min(units, 75) * 5.26
              + min(max(units - 75, 0), 125) * 7.20
              + max(units - 200, 0) * 9.94)
    return round((energy + 42) * (1 + vat_rate), 2)

print(slab_bill(245))                        # 1872.99
print(slab_bill(245, vat_rate=0.075))       # keyword argument – self-documenting
```

Rules that make functions production-grade: **one job per function**; the name is a verb phrase (`calculate_vat`, `send_reminder`); it `return`s data instead of `print`ing it (the caller decides what to do — print, save, send); the `"""docstring"""` states what it returns. A function with no `return` returns `None`.

⚠ WARNING

Never use a mutable default: `def add(item, basket=[])` shares **one** list across every call — items from previous calls leak in. Use `def add(item, basket=None):` then `if basket is None: basket = []`.

বাংলা

ফাংশন হলো নামসহ পুনর্ব্যবহারযোগ্য ব্লক: প্যারামিটার ঢোকে, `return` দিয়ে ফল বেরোয়। পেশাদার নিয়ম: এক ফাংশন = এক কাজ; নাম ক্রিয়াপদে; ফল **return** করুন, print নয় — কী করবে তা কলকারী ঠিক করবে; ডকস্ট্রিং লিখুন। `return` না থাকলে `None` ফেরত আসে। বড় ফাঁদ: ডিফল্ট হিসেবে `[]`-এর মতো পরিবর্তনযোগ্য মান দেবেন না — সব কলে **একই** লিস্ট ভাগাভাগি হয়; বদলে `None` দিয়ে ভেতরে তৈরি করুন।

2.5 *args, **kwargs & Scope

`*args` collects any number of positional arguments into a tuple; `**kwargs` collects keyword arguments into a dict. You meet them constantly in libraries and decorators (6.2):

PYTHON

```
def log_event(event, *details, **meta):
    print(event, details, meta)

log_event("login_failed", "bad password", "3rd attempt", user="rahim", ip="10.0.0.7")
# login_failed ('bad password', '3rd attempt') {'user': 'rahim', 'ip': '10.0.0.7'}
```

Scope: variables created inside a function exist only inside it (local); the function can *read* outer variables but assigning creates a new local one. Production habit: don't mutate globals from inside functions — pass values in, return values out. (`global` exists; treat it as a code smell.)

বাংলা

`*args` যেকোনো সংখ্যক সাধারণ আর্গুমেন্টকে টাপলে জমা করে, `**kwargs` কীওয়ার্ড আর্গুমেন্টকে ডিকশনারিতে — লাইব্রেরি ও ডেকোরেটরে (৬.২) এদের দেখা পাবেন সারাক্ষণ। **স্কোপ:** ফাংশনের ভেতরের ভেরিয়েবল বাইরে নেই; বাইরেরটা পড়া যায়, কিন্তু অ্যাসাইন করলে নতুন লোকাল তৈরি হয়। পেশাদার অভ্যাস: গ্লোবাল বদলাবেন না — মান **ঢোকান** প্যারামিটারে, **ফেরত নিন** `return`-এ।

2.6 Lab: Login Attempt Limiter with Lockout

Real scenario: every serious login form limits attempts to slow down password-guessing (brute-force) attacks. Banks lock after 3 wrong tries. Build the whole flow with only chapters 1–2:

PYTHON

```
# login_guard.py - attempt limiting + lockout, chapter-2 tools only
MAX_ATTEMPTS = 3
CORRECT_PIN = "4got1"          # in real systems: hashed, never hard-coded

def check_pin(entered, correct):
    """Return True if PIN matches (strip spaces, exact compare)."""
    return entered.strip() == correct

attempts = 0
while attempts < MAX_ATTEMPTS:
    pin = input(f"Enter PIN ({MAX_ATTEMPTS - attempts} tries left): ")
    if not pin.strip():
        print("Empty input - not counted."); continue
    attempts += 1
    if check_pin(pin, CORRECT_PIN):
        print("✓ Welcome back."); break
    print("✗ Wrong PIN.")
else:
    # while-else: loop ended WITHOUT break
    print("Account locked for 15 minutes. SMS sent to owner.")
```

OUTPUT

```
Enter PIN (3 tries left): 1111
✗ Wrong PIN.
Enter PIN (2 tries left):
Empty input - not counted.
Enter PIN (2 tries left): 4got1
✓ Welcome back.
```

Notice the architecture, not just the syntax: the *policy* (3 tries) is a named constant at the top; the *check* is a small pure function you could unit-test; `continue` implements "empty input doesn't burn a try"; and `while-else` cleanly separates "succeeded" from "ran out of attempts" — no flag variable needed.

বাংলা

বাস্তব দৃশ্য: ক্রুট-ফোর্স ঠেকাতে প্রতিটি লগইন ফর্ম চেষ্টার সংখ্যা সীমিত করে। গঠনটা লক্ষ করুন: নীতি (৩ বার) উপরে নামকরা কনস্ট্যান্ট; যাচাই আলাদা ছোট ফাংশনে (পরে টেস্ট করা যাবে); খালি ইনপুটে `continue` — চেষ্টা খরচ হয় না; আর `while-else` দিয়ে "সফল" বনাম "সুযোগ শেষ" আলাদা — কোনো অতিরিক্ত ফ্ল্যাগ ভেরিয়েবল লাগেনি। বাস্তবে PIN কখনো কোডে সরাসরি লেখা থাকে না — হ্যাশ করা থাকে।

2.7 Common Mistakes

Mistake	Symptom	Fix
Forgetting <code>:</code> after <code>if</code> / <code>for</code> / <code>def</code>	<code>SyntaxError: expected ':'</code>	Every block header ends with <code>:</code>
Mixing tabs and spaces	<code>IndentationError</code>	Set editor to 4-space indent
<code>while</code> with no progress toward False	Program hangs forever	Ensure counter/condition changes inside
Modifying a list while looping over it	Items silently skipped	Loop over a copy: <code>for x in items[:]:</code>
Calling before defining	<code>NameError</code>	<code>def</code> must run before the call line
Mutable default <code>def f(x, acc=[])</code>	Data leaks between calls	Default <code>None</code> , create inside

বাংলা

ঘন ঘন ভুল: ব্লকের শেষে `:` ভোলা; ট্যাব-স্পেস মেশানো; `while`-এর ভেতরে শর্ত বদলানোর কিছু না রাখা (অনন্ত লুপ); লুপ চলাকালে সেই লিস্টই বদলানো (কপির উপর ঘুরুন: `items[:]`); সংজ্ঞায়িত করার আগেই ফাংশন কল করা; আর মিউটেবল ডিফল্ট।

2.8 Practice Exercises (with Answers)

1. **FizzBuzz, business edition:** for order IDs 1–30, print `COD` if divisible by 3, `Card` if by 5, `COD+Card` if by both, else the ID. (Interviews still ask this.)
2. **Password strength checker:** function returning `"weak"`, `"medium"`, or `"strong"` — length ≥ 8 , has a digit, has an uppercase (use `any(c.isdigit() for c in pw)`).
3. **ATM menu:** `while True` menu — 1) balance 2) deposit 3) withdraw (reject overdraft) 4) quit. Keep each action in its own function.

PYTHON

```
# --- Answers (2 shown; 1 and 3 follow the same patterns) ---
def strength(pw):
    score = sum([len(pw) >= 8,
                 any(c.isdigit() for c in pw),
                 any(c.isupper() for c in pw)])
    return ["weak", "weak", "medium", "strong"][score]

print(strength("abc"))          # weak
print(strength("Dhaka2026"))    # strong

for i in range(1, 31):
    tag = ("COD" if i % 3 == 0 else "") + ("Card" if i % 5 == 0 else "")
    print(tag.replace("CODCard", "COD+Card") or i)
```

বাংলা

অনুশীলন: (১) FizzBuzz-এর ব্যবসায়িক রূপ — ৩-এ বিভাজ্য হলে COD, ৫-এ Card, দুটোতেই COD+Card; (২) পাসওয়ার্ড শক্তি যাচাই — দৈর্ঘ্য, সংখ্যা, বড় হাতের অক্ষর মিলিয়ে স্কোর; (৩) `while True` দিয়ে ATM মেনু, প্রতিটি কাজ আলাদা ফাংশনে। উত্তরে `any(...)` আর `sum([bool, bool, bool])` কৌশল দুটি ভালো করে দেখুন — বাস্তব কোডে খুব চলে।

Data Structures That Run Real Systems

বাস্তব সিস্টেম চালানো ডেটা স্ট্রাকচার

IN THIS CHAPTER · এই অধ্যায়ে

Lists and the copy trap · tuples & unpacking · dicts with the counting and grouping patterns · sets for dedupe & blacklists · comprehensions and key-sorting · Lab: top-selling products report.

3.1 Lists: Ordered, Changeable Collections

Plain-language first: a list is an ordered row of values you can grow, shrink and edit — the workhorse container of Python.

PYTHON

```
queue = ["ORD-101", "ORD-102", "ORD-103"]

queue.append("ORD-104")           # add to end
queue.insert(0, "ORD-100")        # add at position
first = queue.pop(0)              # remove & return front -> "ORD-100"
queue.remove("ORD-102")           # remove by value (first match)
print(len(queue), queue[0], queue[-1]) # 3 ORD-101 ORD-104
print(queue[1:3])                 # slicing works exactly like strings
```

Also essential: `sorted(nums)` (new sorted list) vs `nums.sort()` (in place, returns `None`!), `sum`, `min`, `max`, `"x" in queue`, `queue.count(v)`, `reversed()`.

⚠ WARNING

`b = a` does **not** copy a list — both names point at the same list, so editing `b` edits `a`. Real copy: `b = a.copy()` (or `a[:]`). This single fact causes more silent data corruption than any other beginner topic.

বাংলা

লিস্ট হলো সাজানো, পরিবর্তনযোগ্য সংগ্রহ — Python-এর প্রধান কন্টেইনার। `append`, `insert`, `pop`, `remove`, স্লাইসিং — সবই দেখানো হলো। খেয়াল রাখুন: `nums.sort()` জায়গায় বসে সাজায় আর `None` ফেরত দেয়, `sorted(nums)` নতুন লিস্ট দেয়। **সবচেয়ে বড় ফাঁদ:** `b = a` কপি নয় — দুই নাম একই লিস্টের দিকে; `b` বদলালে `a`-ও বদলে যায়। আসল কপি: `a.copy()`।

3.2 Tuples: Locked Records

A tuple is an immutable list — once made, never edited. Use it for fixed-shape records where each *position* means something: a coordinate, an RGB colour, a (date, amount) pair. Immutability is a feature: nothing downstream can accidentally corrupt it, and tuples can be dict keys.

PYTHON

```
point = (23.8103, 90.4125)           # Dhaka lat, lon
date, amount = ("2026-08-19", 4500) # unpacking — used everywhere
low, high = high, low                # the classic Python swap

def min_max(nums):
    return min(nums), max(nums)       # returning "two values" = one tuple
lo, hi = min_max([4, 9, 1])
```

টাপল হলো অপরিবর্তনীয় লিস্ট — একবার বানাতে আর বদলানো যায় না। নির্দিষ্ট আকারের রেকর্ডে ব্যবহার করুন যেখানে প্রতিটি অবস্থানের অর্থ আছে: স্থানাঙ্ক, (তারিখ, টাকা) জোড়া। অপরিবর্তনীয়তা সুবিধা — ভুলে কেউ নষ্ট করতে পারে না, আর ডিকশনারির কী হিসেবেও চলে।

আনপ্যাকিং (`date, amount = record`) আর `return a, b` দিয়ে দুটি মান ফেরত — এ দুটি রোজকার অস্ত্র।

3.3 Dictionaries: The Most Important Structure in Python

Plain-language first: a dict maps a **key** to a **value**, like a phone book maps names to numbers. Lookup by key is instant regardless of size — this is why dicts run caches, configs, counters, JSON and half of every real codebase.

PYTHON

```
product = {"sku": "SKU-77", "name": "Router AC1200", "price": 3850, "stock": 14}

print(product["price"])           # 3850 — KeyError if key missing
print(product.get("discount", 0)) # 0 — safe lookup with default
product["stock"] -= 1             # update
product["warranty_months"] = 12   # add new key

for key, value in product.items(): # .keys() / .values() / .items()
    print(f"{key:>16}: {value}")
```

The two patterns that appear in virtually every production script:

PYTHON

```
# 1) COUNTING — how many of each?
status_count = {}
for order in ["paid", "cod", "paid", "failed", "paid"]:
    status_count[order] = status_count.get(order, 0) + 1
# {'paid': 3, 'cod': 1, 'failed': 1}

# 2) GROUPING — bucket items by a key
by_city = {}
for name, city in [("Rahim", "Dhaka"), ("Sara", "Khulna"), ("Karim", "Dhaka")]:
    by_city.setdefault(city, []).append(name)
# {'Dhaka': ['Rahim', 'Karim'], 'Khulna': ['Sara']}
```

★ KEY POINT

Memorise `d.get(k, default)` for counting and `d.setdefault(k, []).append(v)` for grouping. These two lines replace thousands of clumsy if-blocks across your career.

ডিকশনারি **কী** → **ভালু** ম্যাপ করে, ফোনবুকের মতো — আকার যত বড়ই হোক, কী দিয়ে খোঁজা তাৎক্ষণিক। তাই ক্যাশ, কনফিগ, কাউন্টার, JSON — সব ডিক্টে চলে। `d[k]` কী না থাকলে এরর দেয়; নিরাপদ পথ `d.get(k, ডিফল্ট)`। দুটি প্যাটার্ন মুখস্থ করুন: **গণনা** — `d[k] = d.get(k, 0) + 1`, আর **গ্রুপিং** — `d.setdefault(k, []).append(v)`। এ দুই লাইন সারাজীবনের হাজারো if-ব্লক বাঁচাবে।

3.4 Sets: Uniqueness & Instant Membership

A set stores unique values, unordered, with instant `in` checks. Two production jobs: **de-duplication** and **fast blacklists**.

PYTHON

```
emails = ["a@x.com", "b@y.com", "a@x.com", "c@z.com", "b@y.com"]
unique = set(emails)           # duplicates gone
print(len(emails), "->", len(unique)) # 5 -> 3

blocked_ips = {"10.0.0.7", "10.0.0.9"} # checking a set is instant
if request_ip in blocked_ips: ...      # a list here gets slower as it grows
```

```
paid = {"u1", "u2", "u3"}
active = {"u2", "u3", "u4"}
print(paid & active)      # {'u2', 'u3'} both (intersection)
print(active - paid)      # {'u4'} active but unpaid (difference)
print(paid | active)      # all of them (union)
```

বাংলা

সেট রাখে কেবল অনন্য মান, ক্রমহীন, আর `in` চেক তাৎক্ষণিক। দুই বাস্তব কাজ: **ডুপ্লিকেট দূর করা** (`set(emails)`) আর **ব্ল্যাকলিস্ট** — লক্ষ আইপির তালিকাতেও চেক সমান দ্রুত (লিস্টে যত বড়, তত ধীর)। সেট-অঙ্ক: `&` উভয়ে আছে, `-` একটাতে আছে অন্যটায় নেই, `|` সব মিলিয়ে — "পেইড কিন্তু ইনঅ্যাক্টিভ কারা" ধরনের প্রশ্ন এক লাইনে।

3.5 Comprehensions & Sorting by Key

A comprehension builds a new list/dict/set from an old one in one readable line — Python's signature move:

PYTHON

```
prices = [1200, 850, 15000, 99]

with_vat = [round(p * 1.05) for p in prices]      # transform
expensive = [p for p in prices if p > 1000]      # filter
labels = {p: ("high" if p > 1000 else "low") for p in prices} # dict comp
```

Sorting anything by anything with `key=`:

PYTHON

```
products = [("Router", 3850), ("Cable", 250), ("Switch", 7900)]

by_price = sorted(products, key=lambda item: item[1], reverse=True)
# [('Switch', 7900), ('Router', 3850), ('Cable', 250)]

names = sorted(["banana", "Apple", "cherry"], key=str.lower) # case-insensitive
```

`lambda item: item[1]` is just a nameless one-line function meaning "sort by field #1". Rule of taste: if a comprehension needs more than one `if` or nests two loops, write a normal loop — clarity beats cleverness.

বাংলা

কমপ্রিহেনশন এক লাইনে পুরনো লিস্ট থেকে নতুন লিস্ট/ডিক্ট বানায়: রূপান্তর `[p*1.05 for p in prices]`, ফিল্টার `[p for p in prices if p > 1000]`। আর `sorted(..., key=lambda x: x[1])` মানে "১ নম্বর ফিল্ড ধরে সাজাও" — `lambda` শুধু নামহীন এক-লাইনের ফাংশন। রুটির নিয়ম: কমপ্রিহেনশন জটিল হয়ে গেলে সাধারণ লুপ লিখুন — চতুরতার চেয়ে স্পষ্টতা দামি।

3.6 Lab: Top-Selling Products from Raw Sales Data

Real scenario: a shop's POS system dumps one line per sale. The owner asks: *total revenue, revenue per product, and the top 3 sellers*. This shape — raw rows → grouped totals → sorted report — is the single most common data task in industry.

PYTHON

```
# sales_report.py
sales = [                                # (product, qty, unit_price) straight from the POS
    ("Router AC1200", 2, 3850), ("Cat6 Cable", 10, 250),
    ("Router AC1200", 1, 3850), ("Switch 8p", 3, 2100),
    ("Cat6 Cable", 25, 250), ("IP Camera", 4, 4500),
    ("Switch 8p", 2, 2100), ("Cat6 Cable", 8, 250),
]

revenue = {}                             # product -> total taka
units = {}                               # product -> total qty
for product, qty, price in sales:        # tuple unpacking in the loop
    revenue[product] = revenue.get(product, 0) + qty * price
    units[product] = units.get(product, 0) + qty
```

```
top3 = sorted(revenue.items(), key=lambda kv: kv[1], reverse=True)[:3]

print(f"Grand total: {sum(revenue.values())}, BDT")
print(f"\n{'#':<3}{ 'Product':<16}{ 'Units':>6}{ 'Revenue':>12}")
for rank, (product, taka) in enumerate(top3, start=1):
    print(f"{rank:<3}{product:<16}{units[product]:>6}{taka:>12,}")
```

OUTPUT

Grand total: 43,000 BDT

#	Product	Units	Revenue
1	IP Camera	4	18,000
2	Router AC1200	3	11,550
3	Cat6 Cable	43	10,750

Every chapter-3 tool is on duty: tuple unpacking in the `for` header, the `get(k, 0)` counting pattern (twice), `dict.items()`, `sorted` with a `lambda` key, slicing `[:3]`, and `enumerate` for ranks. This 20-line shape scales unchanged to a million rows.

বাংলা

বাস্তব দৃশ্য: POS থেকে প্রতি বিক্রয়ে এক লাইন; মালিক চান মোট আয়, পণ্যভিত্তিক আয় আর টপ-৩। কাঁচা সারি → গ্রুপ করে যোগফল → সাজানো রিপোর্ট — শিল্পের সবচেয়ে সাধারণ ডেটা কাজ এটিই। লক্ষ করুন: লুপ হেডারেই টাপল আনপ্যাকিং, দুবার `get(k, 0)` গণনা প্যাটার্ন, `sorted(..., key=lambda kv: kv[1], reverse=True)[:3]` দিয়ে টপ-৩, আর র‍্যাঙ্কে `enumerate`। এই ২০ লাইনের গঠন দশ লাখ সারিতেও অপরিবর্তিত চলে।

3.7 Common Mistakes

Mistake	Symptom	Fix
<code>b = a</code> believing it copies	Editing one changes "both"	<code>b = a.copy()</code>
<code>x = nums.sort()</code>	<code>x</code> is <code>None</code>	<code>sort()</code> is in-place; use <code>sorted()</code> for a new list
<code>d["missing"]</code>	<code>KeyError</code> crash	<code>d.get("missing", default)</code>
Removing items while iterating	Skipped elements	Iterate a copy or build a filtered comprehension
<code>{}</code> to make an empty set	You made an empty dict	Empty set is <code>set()</code>
Using a list for membership checks in a hot loop	Slows as data grows	Convert to a <code>set</code> once

বাংলা

ভুলগুলো: `b = a` -কে কপি ভাবা; `x = nums.sort()` লিখে `None` পাওয়া; না-থাকা কী-তে `d[k]` (ব্যবহার করুন `get`); লুপ চলাকালে সেই লিস্ট থেকে মুছা; খালি সেট বানাতে `{}` লেখা (ওটা ডিক্ট — খালি সেট `set()`); আর বারবার মেম্বারশিপ চেকে লিস্ট ব্যবহার করা (সেটে রূপান্তর করুন)।

3.8 Practice Exercises (with Answers)

- Repeat visitors:** from a list of visitor IDs with duplicates, print how many *distinct* visitors came, and which IDs visited more than once.
- Gradebook:** `{"Rahim": [80, 92], "Sara": [95, 88]}` → print each student's average, then the topper, using `.items()`, `sum`, `len`, and `max(..., key=...)`.
- Invert a dict:** turn `{"dhaka": "01", "khulna": "04"}` into `{"01": "dhaka", ...}` with a dict comprehension.

PYTHON

```
# --- Answers ---
# 1
visits = ["u1", "u2", "u1", "u3", "u2", "u1"]
seen = {}
for v in visits: seen[v] = seen.get(v, 0) + 1
print(len(seen), [u for u, n in seen.items() if n > 1]) # 3 ['u1', 'u2']

# 2
grades = {"Rahim": [80, 92], "Sara": [95, 88]}
avg = {name: sum(marks) / len(marks) for name, marks in grades.items()}
print(avg, "topper:", max(avg, key=avg.get))

# 3
codes = {"dhaka": "01", "khulna": "04"}
print({v: k for k, v in codes.items()})
```

বাংলা

অনুশীলন: (১) ভিজিটর আইডি থেকে স্বতন্ত্র সংখ্যা ও একাধিকবার আসা আইডি; (২) গ্রেডবুক থেকে গড় ও `max(avg, key=avg.get)` দিয়ে টপার; (৩) ডিক্ট কমপ্রিহেনশনে কী-ভ্যালু উল্টানো `{v: k for k, v in d.items()}` — তিনটিই সাক্ষাৎকারে ও বাস্তব কাজে নিয়মিত আসে।

Files, Errors, JSON & CSV

ফাইল, এরর, JSON ও CSV

IN THIS CHAPTER · এই অধ্যায়ে

with-open and modes · try/except/finally done right · raising your own errors (guard clauses) · JSON and CSV the professional way · Lab: server log analyzer that spots an attack.

4.1 Reading & Writing Files

Plain-language first: files let your program remember things after it exits and exchange data with other systems. The `with` statement opens a file and *guarantees* it closes, even if your code crashes mid-way.

PYTHON

```
with open("report.txt", "w", encoding="utf-8") as f:    # w = write (overwrites!)
    f.write("Daily total: 43,000 BDT\n")

with open("report.txt", "a", encoding="utf-8") as f:    # a = append to end
    f.write("Generated by sales_report.py\n")

with open("report.txt", encoding="utf-8") as f:        # default mode: r = read
    for line in f:                                     # memory-safe: one line at a time
        print(line.rstrip())                           # strip the trailing \n
```

Modes: `r` read (error if missing) · `w` write (**silently erases** existing content) · `a` append · `rb` / `wb` for binary (images, PDFs). Always pass `encoding="utf-8"` — required for Bangla text on Windows, harmless everywhere else. For huge files, `for line in f:` streams line-by-line; `f.read()` loads everything into memory at once — fine for small files only.

বাংলা

ফাইল দিয়ে প্রোগ্রাম বন্ধ হওয়ার পরও ডেটা টিকে থাকে। `with open(...) as f:` ব্যবহার করুন — ক্র্যাশ করলেও ফাইল বন্ধ হওয়ার নিশ্চয়তা দেয়। মোড: `r` পড়া, `w` লেখা (**আগের সব মুছে দেয়!**), `a` শেষে যোগ। বাংলা টেক্সটের জন্য `encoding="utf-8"` বাধ্যতামূলক। বড় ফাইলে `for line in f:` দিয়ে লাইন-বাই-লাইন পড়ুন — `f.read()` পুরোটা মেমোরিতে তোলে।

4.2 Errors & try / except / finally

An **exception** is Python's alarm when something goes wrong at runtime. Unhandled, it kills the program. `try/except` says: *attempt this; if that specific alarm rings, do this instead.*

PYTHON

```
def safe_rate(taka, units):
    try:
        return taka / units
    except ZeroDivisionError:
        return 0.0

try:
    with open("config.json") as f:
        raw = f.read()
        limit = int(raw)
except FileNotFoundError:
    print("config.json missing – using defaults"); limit = 100
except ValueError:
    print("config.json is corrupt"); limit = 100
except Exception as e:
    # last-resort net: LOG it, never hide it
    print(f"Unexpected: {type(e).__name__}: {e}"); raise
```

```
finally:
    print("cleanup runs no matter what")    # close connections, release locks
```

Catch **specific** exceptions first; each gets its own recovery. The names you'll meet weekly: `ValueError` (bad conversion), `TypeError` (wrong type), `KeyError` / `IndexError` (missing key/position), `FileNotFoundError`, `ZeroDivisionError`.

⚠ WARNING

A bare `except:` (or `except Exception: pass`) that silently swallows everything is the most dangerous line in Python — bugs vanish without a trace and surface weeks later as corrupt data. Catch what you can *handle*; log and re-`raise` the rest.

বাংলা

রানটাইমে কিছু ভুল হলে Python **এক্সেপশন** তোলে; না ধরলে প্রোগ্রাম মরে যায়। `try/except` মানে: "এটা চেষ্টা করো; ওই নির্দিষ্ট অ্যালার্ম বাজলে বরং এটা করো।" নির্দিষ্ট এক্সেপশন আগে ধরুন — প্রতিটির নিজস্ব সমাধান। `finally` চলে সব অবস্থাতেই — কানেকশন বন্ধ, লক ছাড়ার জায়গা। **সবচেয়ে বিপজ্জনক লাইন:** খালি `except: pass` — বাগ নিঃশব্দে হারিয়ে যায়, সপ্তাহ পরে নষ্ট ডেটা হয়ে ফেরে। যা সামলাতে পারেন তা-ই ধরুন; বাকিটা লগ করে `raise` করুন।

4.3 Raising Your Own Errors

Good production code *refuses bad input loudly and early* instead of computing garbage quietly:

PYTHON

```
class InsufficientStock(Exception):
    """Raised when an order exceeds available stock."""

def reserve(stock, qty):
    if qty <= 0:
        raise ValueError(f"qty must be positive, got {qty}")
    if qty > stock:
        raise InsufficientStock(f"want {qty}, only {stock} left")
    return stock - qty

try:
    reserve(stock=5, qty=8)
except InsufficientStock as e:
    print("Backorder created:", e)
```

`raise` fires the alarm yourself; a two-line custom exception class gives the failure a *name* callers can catch precisely, instead of string-matching generic errors. This validate-at-the-door pattern is called a **guard clause**.

বাংলা

ভালো প্রোডাকশন কোড খারাপ ইনপুট পেলে চুপচাপ ভুল হিসাব না করে **শুরুতেই জোরে আপত্তি** করে — একে বলে গার্ড ক্লজ। `raise` দিয়ে নিজেই অ্যালার্ম বাজান; দুই লাইনের কাস্টম এক্সেপশন ক্লাস (`class InsufficientStock(Exception)`) ব্যর্থতাকে একটা **নাম** দেয়, যা কলকারী নিখুঁতভাবে ধরতে পারে।

4.4 JSON: How Programs Talk to Each Other

JSON (JavaScript Object Notation) is the universal text format for structured data — every web API, config file and inter-service message uses it. The magic: JSON objects/arrays map exactly onto Python dicts/lists.

PYTHON

```
import json

settings = {"shop": "TechPoint", "vat": 0.05,
            "branches": ["Dhaka", "Chattogram"], "open": True}

with open("settings.json", "w", encoding="utf-8") as f:
```



```

json.dump(settings, f, indent=2, ensure_ascii=False) # Python -> file

with open("settings.json", encoding="utf-8") as f:
    cfg = json.load(f) # file -> Python
print(cfg["branches"][0]) # Dhaka

text = json.dumps(settings) # -> string (dumps/loads = string versions)
back = json.loads(text) # string -> dict (this is what APIs give you)

```

Memory hook: **dump = Python → JSON out, load = JSON in → Python**; the `s` suffix means *string* instead of file. `indent=2` makes files human-readable; `ensure_ascii=False` keeps বাংলা readable instead of `\u09ac...` escapes.

বাংলা

JSON হলো স্ট্রাকচার্ড ডেটার সর্বজনীন টেক্সট ফরম্যাট — প্রতিটি ওয়েব API ও কনফিগ ফাইল এতে চলে, আর এটা হুবহু Python-এর ডিক্ট/লিস্টে ম্যাপ হয়। মনে রাখার সূত্র: **dump** = Python থেকে বাইরে, **load** = ভেতরে; শেষে `s` থাকলে ফাইল নয়, স্ট্রিং। বাংলা পড়ার যোগ্য রাখতে `ensure_ascii=False` দিন, মানুষের পড়ার জন্য `indent=2`।

4.5 CSV: The Language of Spreadsheets

CSV (Comma-Separated Values) is how Excel, banks and ERPs export tables. Never split CSV by hand with `.split(",")` — real data has commas *inside* quoted fields (`"Dhaka, Bangladesh"`). The `csv` module handles all of it:

PYTHON

```

import csv

with open("orders.csv", encoding="utf-8", newline="") as f:
    for row in csv.DictReader(f): # each row -> dict keyed by header
        print(row["customer"], row["amount"])

rows = [{"customer": "Karim Store", "amount": 4500},
        {"customer": "Ayesha Traders", "amount": 12800}]
with open("out.csv", "w", encoding="utf-8", newline="") as f:
    w = csv.DictWriter(f, fieldnames=["customer", "amount"])
    w.writeheader(); w.writerows(rows)

```

Two production notes: `newline=""` prevents blank lines on Windows, and **every CSV value arrives as a string** — convert `int(row["amount"])` before math (same trap as `input()`).

বাংলা

CSV হলো Excel/ব্যাংক/ERP-এর টেবিল ফরম্যাট। কখনোই হাতে `.split(",")` করবেন না — কোটেশনের ভেতরের কমা (`"Dhaka, Bangladesh"`) সব ভেঙে দেবে; `csv` মডিউল ব্যবহার করুন। `DictReader` প্রতি সারিকে হেডার-কী-ওয়ালা ডিক্ট বানায়। দুটি নোট: Windows-এ ফাঁকা লাইন এড়াতে `newline=""` , আর CSV-র প্রতিটি মান স্ট্রিং — অঙ্কের আগে `int()` / `float()` করুন।

4.6 Lab: Server Log Analyzer (404s & Top IPs)

Real scenario: your website misbehaves. The nginx access log has 200,000 lines. Ops asks: *how many errors, which URLs 404 the most, and which IPs hammer us hardest?* — the daily bread of DevOps, SRE and security work.

PYTHON

```

# log_analyzer.py – everything from chapters 1-4 working together
LOG = """10.0.0.7 GET /login 200
10.0.0.9 GET /admin 404
10.0.0.7 POST /login 401
10.0.0.9 GET /admin.php 404
10.0.0.9 GET /wp-login.php 404
10.0.0.5 GET /products 200
10.0.0.9 GET /.env 404
10.0.0.7 GET /products 200"""
with open("access.log", "w") as f: # create sample data
    f.write(LOG)

```

```

status_count, not_found, hits_by_ip = {}, {}, {}

with open("access.log") as f:
    for lineno, line in enumerate(f, start=1):
        parts = line.split()
        if len(parts) != 4:
            print(f"skip malformed line {lineno}"); continue # real logs are dirty
        ip, method, url, status = parts
        status_count[status] = status_count.get(status, 0) + 1
        hits_by_ip[ip] = hits_by_ip.get(ip, 0) + 1
        if status == "404":
            not_found[url] = not_found.get(url, 0) + 1

print("Status summary:", status_count)
print("Top 404 URLs :", sorted(not_found.items(), key=lambda kv: kv[1], reverse=True)[:3])
print("Top IPs      :", sorted(hits_by_ip.items(), key=lambda kv: kv[1], reverse=True)[:3])

import json
with open("log_summary.json", "w") as f: # hand results to other tools
    json.dump({"status": status_count, "top_404": not_found,
              "by_ip": hits_by_ip}, f, indent=2)

```

OUTPUT

```

Status summary: {'200': 3, '404': 4, '401': 1}
Top 404 URLs : [('/admin', 1), ('/admin.php', 1), ('/wp-login.php', 1)]
Top IPs      : [('10.0.0.9', 4), ('10.0.0.7', 3), ('10.0.0.5', 1)]

```

Read the story in the output: one IP producing many 404s on `/admin.php`, `/wp-login.php`, `/.env` is a **vulnerability scanner** probing your server — this tiny script is genuinely how such attacks get spotted. Note the defensive `len(parts) != 4` check: production data is *always* dirtier than you expect, and one malformed line must not kill a 200,000-line job.

বাংলা

বাস্তব দৃশ্য: ২ লাখ লাইনের সার্ভার লগ থেকে বের করতে হবে — কত এরর, কোন URL-এ বেশি 404, কোন আইপি সবচেয়ে বেশি আঘাত করছে। আউটপুটের গল্পটা দেখুন: এক আইপি থেকে `/admin.php`, `/wp-login.php`, `/.env`-এ পরপর 404 মানে **ভালনারেবিলিটি স্ক্যানার** আপনার সার্ভার হাতড়াচ্ছে — সত্যিকারের আক্রমণ এভাবেই ধরা পড়ে। আর `len(parts) != 4` চেকটি লক্ষ করুন: প্রোডাকশনের ডেটা সবসময় নোংরা; একটি ভাঙা লাইন যেন পুরো কাজ না মারে।

4.7 Common Mistakes

Mistake	Symptom	Fix
Opening with <code>"w"</code> to "update"	File wiped empty	<code>"a"</code> to append, or read-modify-write
No <code>encoding="utf-8"</code>	Bangla becomes <code>à ¬...</code> mojibake	Always pass it, read <i>and</i> write
<code>except: pass</code>	Bugs disappear silently	Catch specific types; log the rest
<code>json.load</code> vs <code>json.loads</code> mixed up	<code>TypeError</code> about file/str	No <code>s</code> = file, <code>s</code> = string
Math on CSV values	<code>"4500" * 2 = "45004500"</code>	Convert: <code>float(row["amount"])</code>
Forgetting <code>newline=""</code> when writing CSV	Blank line between rows (Windows)	Add it to <code>open()</code>

বাংলা

ভুলসমূহ: "আপডেট" করতে `"w"` মোডে খোলা (ফাইল মুছে যায়); `encoding="utf-8"` বাদ দেওয়া (বাংলা ভেঙে যায়); `except: pass` দিয়ে বাগ গিলে ফেলা; `load` / `loads` গুলিয়ে ফেলা (`s` = স্ট্রিং); CSV-র স্ট্রিং মানে সরাসরি অঙ্ক; CSV লেখায় `newline=""` ভোলা।

4.8 Practice Exercises (with Answers)

1. **Robust average:** read `marks.txt` (one number per line, some lines garbage); print the average of valid lines and how many lines you skipped.
2. **Config with fallback:** `load_config(path)` returns the parsed JSON, or `{"theme": "light", "retries": 3}` if the file is missing or corrupt.
3. **CSV → JSON converter:** read `orders.csv`, convert `amount` to float, write `orders.json` — the classic glue script.

PYTHON

```
# --- Answer 2 (the pattern the others reuse) ---
import json

def load_config(path):
    defaults = {"theme": "light", "retries": 3}
    try:
        with open(path, encoding="utf-8") as f:
            return {**defaults, **json.load(f)} # file values override defaults
    except (FileNotFoundError, json.JSONDecodeError):
        return defaults
```

The `{**defaults, **loaded}` merge — defaults first, real values on top — is exactly how mature applications layer configuration.

বাংলা

অনুশীলন: (১) নোংরা লাইনসহ ফাইল থেকে নিরাপদ গড়; (২) কনফিগ লোডার — ফাইল না থাকলে বা নষ্ট হলে ডিফল্ট ফেরত; (৩) CSV → JSON রূপান্তরকারী। উত্তরের `{**defaults, **loaded}` মার্জটি খেয়াল করুন — আগে ডিফল্ট, উপরে আসল মান — পরিণত অ্যাপ্লিকেশন কনফিগ ঠিক এভাবেই স্তরে স্তরে সাজায়।

Object-Oriented Python & Project Structure

অবজেক্ট-ওরিয়েন্টেড পাইথন ও প্রজেক্ট গঠন

IN THIS CHAPTER · এই অধ্যায়ে

Classes, self and `__init__` · `__repr__`/`__eq__` · inheritance vs composition · `@dataclass` and `@property` · modules and the `__main__` guard · Lab: inventory manager with low-stock alerts.

5.1 Classes & Objects: Data + Behaviour Together

Plain-language first: a class is a blueprint; an object is one real thing built from it. Instead of juggling a dict of fields *and* loose functions that operate on it, a class bundles both — the data and the operations that belong to it.

Until now, an "account" was a dict, and `deposit()` was a function somewhere hoping to receive the right dict. Object-Oriented Programming (OOP) glues them:

PYTHON

```
class Account:
    def __init__(self, owner, balance=0):  # runs when the object is created
        self.owner = owner                # self = "this particular object"
        self.balance = balance

    def deposit(self, amount):
        if amount <= 0:
            raise ValueError("deposit must be positive")
        self.balance += amount

    def withdraw(self, amount):
        if amount > self.balance:
            raise ValueError("insufficient funds")
        self.balance -= amount
        return amount

acc = Account("Rahim", 1000)  # __init__ runs; acc is an object (instance)
acc.deposit(500)
acc.withdraw(300)
print(acc.owner, acc.balance)  # Rahim 1200
```

`self` is simply "the object this method was called on" — `acc.deposit(500)` secretly means `Account.deposit(acc, 500)`. Each object carries its own data: a second `Account("Sara")` has a completely separate balance.

বাংলা

ক্লাস হলো নকশা, অবজেক্ট সেই নকশা থেকে বানানো একেকটি বাস্তব জিনিস। এতদিন "অ্যাকাউন্ট" ছিল একটি ডিক্ট আর `deposit()` ছিল দূরের কোনো ফাংশন — OOP এ দুটোকে একসাথে বাঁধে। `__init__` চলে অবজেক্ট তৈরির মুহুর্তে; `self` মানে "এই নির্দিষ্ট অবজেক্টটি" — `acc.deposit(500)` আসলে `Account.deposit(acc, 500)`। প্রতিটি অবজেক্টের নিজস্ব ডেটা: রহিমের ব্যালান্স আর সারার ব্যালান্স সম্পূর্ণ আলাদা।

5.2 `__repr__`, `__eq__` & Other Dunder Methods

Double-underscore ("dunder") methods let your objects plug into Python's own syntax — printing, `==`, `len()`, sorting:

PYTHON

```
class Product:
    def __init__(self, sku, price):
        self.sku, self.price = sku, price
```

```
def __repr__(self):
    # what print() / debugger shows
    return f"Product({self.sku!r}, {self.price})"

def __eq__(self, other):
    # defines ==
    return isinstance(other, Product) and self.sku == other.sku

p = Product("SKU-77", 3850)
print(p)
print(p == Product("SKU-77", 9999))
```

Without `__repr__`, printing shows the useless `<__main__.Product object at 0x7f...>`. Define `__repr__` on every class you write — future-you debugging at midnight will be grateful.

বাংলা

ডাবল-আন্ডারস্কোর ("ডাবলার") মেথড দিয়ে আপনার অবজেক্ট Python-এর নিজস্ব সিনট্যাক্সে যুক্ত হয়: `__repr__` ঠিক করে `print()` কী দেখাবে, `__eq__` ঠিক করে `==` কীভাবে তুলনা করবে। `__repr__` না লিখলে প্রিন্টে অকেজো `<object at 0x...>` দেখায় — তাই প্রতিটি ক্লাসে `__repr__` লিখুন; মাঝরাতে ডিবাগ করার সময় নিজেকেই ধন্যবাদ দেবেন।

5.3 Inheritance & Composition

Inheritance — a child class *is a kind of* the parent, keeping its behaviour and overriding what differs:

PYTHON

```
class SavingsAccount(Account):
    # SavingsAccount IS-A Account
    def __init__(self, owner, balance=0, rate=0.06):
        super().__init__(owner, balance)
        # run the parent's setup first
        self.rate = rate

    def add_interest(self):
        # new capability
        self.balance += self.balance * self.rate

    def withdraw(self, amount):
        # override: add a policy
        if amount > 50_000:
            raise ValueError("savings withdrawal cap is 50,000/day")
        return super().withdraw(amount)
        # then reuse the parent logic
```

Composition — an object *has* other objects. A `Shop` has a list of `Product`s; it isn't a kind of product. Industry guidance: prefer composition for most designs, keep inheritance for genuine is-a relationships, and keep hierarchies shallow (one, maybe two levels). Deep inheritance trees are a well-known maintenance trap.

বাংলা

ইনহেরিটেন্স: চাইল্ড ক্লাস প্যারেন্টের এক প্রকার — আচরণ পায়, যা ভিন্ন তা override করে; `super()` দিয়ে প্যারেন্টের কোড পুনর্ব্যবহার করে। **কম্পোজিশন:** একটি অবজেক্টের ভেতরে অন্য অবজেক্ট থাকে — `Shop`-এর আছে অনেক `Product`। শিল্পের পরামর্শ: বেশিরভাগ ডিজাইনে কম্পোজিশন বেছে নিন; ইনহেরিটেন্স রাখুন প্রকৃত "is-a" সম্পর্কে, এবং সর্বোচ্চ এক-দুই স্তর — গভীর ইনহেরিটেন্স-গাছ রক্ষণাবেক্ষণের ফাঁদ।

5.4 @dataclass & @property: Modern Class Writing

Most real classes are mostly *data*. `@dataclass` writes `__init__`, `__repr__` and `__eq__` for you from the field list:

PYTHON

```
from dataclasses import dataclass, field

@dataclass
class Order:
    order_id: str
    customer: str
    amount: float
    items: list = field(default_factory=list)
    # safe mutable default
    status: str = "pending"
```

```
o = Order("ORD-101", "Karim Store", 4500.0)
print(o)           # Order(order_id='ORD-101', customer='Karim Store', ...)
```

`@property` makes a method look like a read-only attribute — perfect for computed values:

PYTHON

```
@dataclass
class Invoice:
    subtotal: float
    vat_rate: float = 0.05

    @property
    def total(self):
        return round(self.subtotal * (1 + self.vat_rate), 2)

print(Invoice(1000).total)    # 1050.0 — no parentheses; can't be assigned
```

Modern production Python is full of dataclasses — they remove the boilerplate that made OOP feel heavy.

বাংলা

বাস্তবের বেশিরভাগ ক্লাস মূলত ডেটা। `@dataclass` ফিল্ডের তালিকা থেকে `__init__`, `__repr__`, `__eq__` নিজে লিখে দেয় — বয়লারপ্লেট শেষ। মিউটেবল ডিফল্টের নিরাপদ রূপ: `field(default_factory=list)`। আর `@property` একটি মেথডকে read-only অ্যাট্রিবিউটের মতো দেখায় — হিসাব করা মান (যেমন VAT-সহ মোট) রাখার আদর্শ জায়গা: `invoice.total`, বন্ধনী ছাড়া।

5.5 Modules, Packages & the `__main__` Guard

A module is just a `.py` file; `import` lets files use each other. Split a growing script by *responsibility*:

OUTPUT

```
shop/
├── models.py    # Product, Order dataclasses
├── billing.py   # slab_bill(), apply_vat()
└── main.py     # entry point — wires everything together
```

PYTHON

```
# main.py
from models import Product
from billing import slab_bill

def run():
    print(slab_bill(245))

if __name__ == "__main__":
    run()           # True only when THIS file is executed directly
                  # skipped when main.py is merely imported
```

The `if __name__ == "__main__":` guard means a file can be both an importable library *and* a runnable script — and imports never trigger surprise side effects. Every professional Python file with runnable code ends this way.

বাংলা

মডিউল মানে একটি `.py` ফাইল; `import` দিয়ে ফাইলগুলো একে অপরকে ব্যবহার করে। বাড়ন্ত স্ক্রিপ্টকে দায়িত্ব অনুযায়ী ভাগ করুন: `models`, `billing`, `main`। `if __name__ == "__main__":` গার্ডের অর্থ: ব্লকটি চলবে কেবল ফাইলটি সরাসরি রান করলে — কেউ `import` করলে নয়। এতে একই ফাইল লাইব্রেরিও, রানযোগ্য স্ক্রিপ্টও। পেশাদার প্রতিটি Python ফাইল এই গার্ড দিয়েই শেষ হয়।

5.6 Lab: Inventory Manager with Low-Stock Alerts

Real scenario: a distributor needs to record sales, block overselling, flag items running low, and report stock value — the beating heart of every POS/ERP system, in 45 lines:

PYTHON

```

# inventory.py
from dataclasses import dataclass

class OutOfStock(Exception): pass

@dataclass
class Item:
    sku: str
    name: str
    price: float
    qty: int
    reorder_at: int = 5

    @property
    def low(self):
        return self.qty <= self.reorder_at

class Inventory:
    def __init__(self):
        self._items = {}  # sku -> Item (dict for instant lookup)

    def add(self, item):
        self._items[item.sku] = item

    def sell(self, sku, qty):
        item = self._items.get(sku)
        if item is None:
            raise KeyError(f"unknown SKU {sku}")
        if qty > item.qty:
            raise OutOfStock(f"{item.name}: want {qty}, have {item.qty}")
        item.qty -= qty
        return qty * item.price

    @property
    def stock_value(self):
        return sum(i.price * i.qty for i in self._items.values())

    def low_stock(self):
        return [i for i in self._items.values() if i.low]

inv = Inventory()
inv.add(Item("R-77", "Router AC1200", 3850, 8))
inv.add(Item("C-06", "Cat6 Cable", 250, 60))
inv.add(Item("S-08", "Switch 8p", 2100, 4))

print("Sold:", inv.sell("R-77", 5), "BDT")
try:
    inv.sell("S-08", 10)
except OutOfStock as e:
    print("Blocked:", e)
print("Stock value:", f"{inv.stock_value:,.0f}")
print("Reorder:", [i.name for i in inv.low_stock()])

```

OUTPUT

```

Sold: 19250.0 BDT
Blocked: Switch 8p: want 10, have 4
Stock value: 34,950
Reorder: ['Router AC1200', 'Switch 8p']

```

Design decisions worth absorbing: `Item` is a dataclass (pure data + one computed property); `Inventory` owns the rules; storage is a dict keyed by SKU (3.3's instant lookup, deliberately chosen); the leading underscore in `_items` signals "internal — go through my methods"; and overselling is *impossible by construction*, not by hoping callers check first.

বাস্তব দৃশ্য: ডিস্ট্রিবিউটরের দরকার — বিক্রি রেকর্ড, ওভারসেল ব্লক, লো-স্টক সংকেত, স্টক-ভ্যালু রিপোর্ট। ডিজাইনটা শিখুন: `Item` ডেটাক্লাস (শুধু ডেটা + একটি হিসাবি property); নিয়মকানুনের মালিক `Inventory`; সংরক্ষণ SKU-কী-ওয়ালা ডিক্টে (তাৎক্ষণিক লুকআপ); `__items__`-এর আন্ডারস্কোর মানে "ভেতরের জিনিস — মেথড দিয়ে ঢুকুন"; আর ওভারসেলিং *গঠনগতভাবেই অসম্ভব* — কলকারীর সততার উপর ভরসা নয়।

5.7 Common Mistakes

Mistake	Symptom	Fix
Forgetting <code>self</code> in a method definition	<code>takes 1 positional argument but 2 were given</code>	First parameter is always <code>self</code>
Forgetting <code>self.</code> when using attributes	<code>NameError: name 'balance' is not defined</code>	<code>self.balance</code> , not <code>balance</code>
Class-level mutable: <code>items = []</code> in class body	All objects share one list	Set it in <code>__init__</code> / <code>default_factory</code>
Skipping <code>super().__init__()</code> in a child	Parent attributes missing	Call it first in child <code>__init__</code>
Classes for everything	200-line class doing one function's job	A plain function is often the right tool

ভুলসমূহ: মেথড সংজ্ঞায় `self` ভোলা; অ্যাট্রিবিউটে `self.` ভোলা; ক্লাস-বডিতে `items = []` লেখা (সব অবজেক্ট এক লিস্ট ভাগ করে — `__init__`-এ দিন); চাইল্ডে `super().__init__()` না ডাকা; আর সব কিছুতে ক্লাস বানানো — অনেক সময় সাধারণ ফাংশনই সঠিক হাতিয়ার।

5.8 Practice Exercises (with Answers)

- Ride fare:** `Ride` dataclass with `km`, `minutes`; property `fare` = 40 base + 25/km + 2/min. Add `SharedRide(Ride)` overriding `fare` at 60%.
- Library:** `Book` dataclass + `Library` class with `borrow(isbn)` / `return_(isbn)` raising a custom `AlreadyBorrowed` error.
- Split exercise 2 into `models.py` and `main.py` with a `__main__` guard.

PYTHON

```
# --- Answer 1 ---
from dataclasses import dataclass

@dataclass
class Ride:
    km: float
    minutes: float
    @property
    def fare(self):
        return round(40 + 25 * self.km + 2 * self.minutes)

class SharedRide(Ride):
    @property
    def fare(self):
        return round(super().fare * 0.6)

print(Ride(8, 22).fare, SharedRide(8, 22).fare)    # 284 170
```


অনুশীলন: (১) `Ride` ডেটাক্লাস — property দিয়ে ভাড়ার হিসাব, `SharedRide` চাইল্ডে override করে ৬০%; (২) `Library` ক্লাস — কাস্টম `AlreadyBorrowed` এক্সেপশনসহ borrow/return; (৩) কোডটি `models.py` ও `main.py`-তে ভাগ করে `__main__` গার্ড বসানো। উত্তরে দেখুন কীভাবে চাইল্ডের property ভেতর থেকেই `super().fare` পুনর্ব্যবহার করছে।

Advanced Python: Elegant & Efficient Code

অ্যাডভান্সড পাইথন: মার্জিত ও দক্ষ কোড

IN THIS CHAPTER · এই অধ্যায়ে

Generators for gigabyte files · decorators from scratch · custom context managers · type hints · lru_cache, groupby, islice · Lab: retry-with-backoff decorator + streaming pipeline.

6.1 Generators: Process Millions of Rows in Kilobytes of RAM

Plain-language first: a normal function cooks the whole feast at once and hands you everything; a generator hands you one plate at a time, cooking only when you ask.

Replace `return` with `yield` and the function becomes a generator: it pauses at each `yield`, resumes on the next request, and never holds the full dataset in memory.

PYTHON

```
def read_paid_orders(path):
    """Stream matching lines from a file of ANY size."""
    with open(path) as f:
        for line in f:
            if ",paid," in line:
                yield line.rstrip()          # hand out one row, then pause

total = 0
for row in read_paid_orders("orders_2026.csv"): # 10 GB file? No problem.
    total += float(row.split(",")[2])
```

A list version of this loads all 10 GB into RAM and dies; the generator uses a few KB regardless of file size. Generator *expressions* look like comprehensions with `()` and feed aggregations without building a list:

PYTHON

```
big_total = sum(qty * price for _, qty, price in sales) # no list ever exists
first_bad = next((u for u in users if not u["email"]), None) # first match or None
```

★ KEY POINT

Rule of thumb: building a list just to loop over it once? Use a generator. Chains like *read* → *filter* → *transform* → *aggregate* stream beautifully, one item flowing through the whole pipeline at a time.

বাংলা

সাধারণ ফাংশন পুরো ভোজ একবারে রেঁধে দেয়; জেনারেটর চাইলে-তবেই এক প্লেট করে দেয়। `return`-এর জায়গায় `yield` লিখলেই ফাংশন জেনারেটর — প্রতি `yield`-এ থামে, পরের চাহিদায় আবার চলে, পুরো ডেটা কখনোই মেমোরিতে রাখে না। ১০ GB ফাইল লিস্ট তুললে RAM শেষ; জেনারেটরে কয়েক KB-তেই চলে। `sum(x*y for ...)` ধাঁচের জেনারেটর এক্সপ্রেশনে লিস্ট না বানিয়েই যোগফল হয়। নিয়ম: একবার ঘোরার জন্য লিস্ট বানাচ্ছেন? জেনারেটর ব্যবহার করুন।

6.2 Decorators: Wrap Extra Behaviour Around Any Function

Plain-language first: a decorator is gift-wrap for a function — the gift is unchanged, but now something extra happens when it's opened (timing, logging, retrying, caching, auth checks).

It works because functions are values: they can be passed in and returned. A decorator takes a function and returns a wrapped version:

PYTHON

```
import functools, time

def timed(func):
    @functools.wraps(func)           # keep the original name/docstring
    def wrapper(*args, **kwargs):    # accept anything (2.5!)
        t0 = time.perf_counter()
        result = func(*args, **kwargs) # run the real function
        print(f"{func.__name__} took {time.perf_counter() - t0:.3f}s")
        return result
    return wrapper

@timed                               # sugar for: fetch = timed(fetch)
def fetch_report(n):
    time.sleep(0.2); return f"{n} rows"

fetch_report(500)                    # fetch_report took 0.200s
```

The pattern to internalise: outer function receives `func`; inner `wrapper(*args, **kwargs)` does *before* things, calls `func`, does *after* things, returns the result; `@functools.wraps` preserves identity. This is how Flask routes (`@app.route`), pytest fixtures and permission checks all work.

বাংলা

ডেকোরেটর হলো ফাংশনের গিফট-র্যাপ — ভেতরের ফাংশন অপরিবর্তিত, কিন্তু চালানোর সময় বাড়তি কিছু ঘটে (টাইমিং, লগিং, রিট্রাই, ক্যাশিং)। কার্যামো মুখস্থ করুন: বাইরের ফাংশন `func` নেয় → ভেতরের `wrapper(*args, **kwargs)` আগে-কাজ করে, `func` চালায়, পরে-কাজ করে, ফল ফেরত দেয় → `@functools.wraps` আসল নাম রক্ষা করে। `@timed` লেখা মানে `fetch = timed(fetch)`। Flask-এর `@app.route`, pytest — সবই এই কৌশল।

6.3 Context Managers: Guaranteed Setup & Cleanup

You already use one daily: `with open(...)`. The `with` statement guarantees cleanup runs even if the body crashes — files close, locks release, connections return to the pool. Write your own with one decorator:

PYTHON

```
from contextlib import contextmanager
import time

@contextmanager
def timer(label):
    t0 = time.perf_counter()
    try:
        yield                               # ---- the with-body runs here ----
    finally:
        print(f"[{label}] {time.perf_counter() - t0:.3f}s")

with timer("db-export"):
    time.sleep(0.15)                        # even if this raises, timing prints
# [db-export] 0.150s
```

Everything before `yield` is setup, everything in `finally` is teardown. Real uses: temporary directory that always gets deleted, database transaction that commits on success / rolls back on error, "maintenance mode on → off".

বাংলা

`with open(...)` আসলে একটি কনটেক্সট ম্যানেজার — ভেতরের কোড ক্র্যাশ করলেও পরিষ্কার-পরিচ্ছন্নতা (ফাইল বন্ধ, লক মুক্তি) চলেবেই। `@contextmanager` দিয়ে নিজেরটা লিখুন: `yield`-এর আগে সেটআপ, `finally`-তে টিয়ারডাউন, মাঝে `with`-বডি চলে। বাস্তব ব্যবহার: ডাটাবেস ট্রানজ্যাকশন — সফলে commit, এররে rollback; অস্থায়ী ফোল্ডার যা সবসময় মুছে যায়।

6.4 Type Hints: Executable Documentation

Type hints declare what goes in and out. Python doesn't enforce them at runtime — but your editor autocompletes better, tools like `mypy` catch bugs before running, and teammates read your intent instantly. Every modern codebase requires them.

PYTHON

```
def apply_discount(price: float, percent: float = 10.0) -> float:
    return round(price * (1 - percent / 100), 2)

def top_skus(orders: list[dict[str, int]], limit: int = 3) -> list[str]: ...

from typing import Optional
def find_user(uid: str) -> Optional[dict]: # returns dict OR None
    ...
```

Read `->` as "returns". `list[str]`, `dict[str, float]`, `tuple[int, int]` describe containers; `Optional[X]` (or `X | None` in modern style) flags "may be None — caller must check". Start by annotating function signatures only; that's 90% of the value.

বাংলা

টাইপ হিন্ট বলে দেয় কী ঢুকবে, কী বেরোবে: `def f(price: float) -> float`। Python রানটাইমে জোর করে না, কিন্তু এডিটরের অটোমপ্লিট ভালো হয়, `mypy` চালানোর আগেই বাগ ধরে, সহকর্মী এক নজরে উদ্দেশ্য বোঝে — আধুনিক প্রতিটি কোডবেসে বাধ্যতামূলক। `Optional[dict]` বা `dict | None` মানে "None-ও আসতে পারে — কলকারী চেক করবে"। শুরুতে শুধু ফাংশন সিগনেচারে দিন — ৯০% লাভ ওখানেই।

6.5 functools & itertools Essentials

Two standard-library toolboxes that separate intermediate from advanced code:

PYTHON

```
from functools import lru_cache
from itertools import groupby, islice, chain

@lru_cache(maxsize=256) # memoise: cache results by arguments
def shipping_cost(city: str) -> int:
    print("expensive lookup...") # runs once per distinct city
    return 120 if city == "Dhaka" else 180

shipping_cost("Dhaka"); shipping_cost("Dhaka") # lookup printed only once

rows = sorted(sales, key=lambda r: r[0]) # groupby REQUIRES sorted input!
for product, grp in groupby(rows, key=lambda r: r[0]):
    print(product, sum(r[1] for r in grp))

first_100 = islice(read_paid_orders("big.csv"), 100) # slice a generator
everything = chain(list_a, list_b) # loop two sources as one
```

`lru_cache` is the one-line answer to "this pure function is called repeatedly with the same arguments" — instant, dramatic speedups on lookups and recursive computations.

বাংলা

দুটি স্ট্যান্ডার্ড টুলবক্স: `functools.lru_cache` — একই আর্গুমেন্টে বারবার ডাকা ব্যাবহুল ফাংশনের ফল ক্যাশ করে, এক লাইনে নাটকীয় গতি; `itertools.groupby` — সাজানো ডেটাকে দলে ভাগ করে (আগে `sort` বাধ্যতামূলক, নইলে ভুল দল); `islice` জেনারেটর থেকে প্রথম n-টি নেয়; `chain` একাধিক উৎসকে এক লুপে জোড়ে।

6.6 Lab: Retry Decorator + Streaming Pipeline

Real scenario: your nightly job calls a flaky payment-gateway API that randomly times out. Ops requirement: *retry 3 times with growing delays, log each attempt, and never load the whole transaction file into memory*. This combines everything in the chapter:

PYTHON

```
# resilient.py
import functools, random, time

def retry(times: int = 3, delay: float = 0.5):
    """Retry decorator with exponential backoff: delay, 2x, 4x..."""
    def decorator(func):
        @functools.wraps(func)
        def wrapper(*args, **kwargs):
            wait = delay
            for attempt in range(1, times + 1):
                try:
                    return func(*args, **kwargs)
                except ConnectionError as e:
                    if attempt == times:
                        raise # out of retries: escalate
                    print(f"attempt {attempt} failed ({e}); retry in {wait}s")
                    time.sleep(wait)
                    wait *= 2 # exponential backoff
            return wrapper
        return decorator

@retry(times=3, delay=0.2)
def charge(order_id: str, amount: float) -> str:
    if random.random() < 0.6: # simulate 60% flakiness
        raise ConnectionError("gateway timeout")
    return f"{order_id}: charged {amount}"

def stream_orders(path: str):
    """Generator: yield (order_id, amount) without loading the file."""
    with open(path) as f:
        for line in f:
            oid, amount = line.strip().split(",")
            yield oid, float(amount)

with open("txns.csv", "w") as f:
    f.write("ORD-1,4500\nORD-2,1200\nORD-3,880\n")

for oid, amount in stream_orders("txns.csv"):
    print(charge(oid, amount))
```

OUTPUT

```
attempt 1 failed (gateway timeout); retry in 0.2s
ORD-1: charged 4500.0
ORD-2: charged 1200.0
attempt 1 failed (gateway timeout); retry in 0.4s
attempt 2 failed (gateway timeout); retry in 0.8s
ORD-3: charged 880.0
```

Note `retry(times=3)` is a decorator **factory** — a function returning a decorator, which is why there are three nested levels. Exponential backoff (0.2 → 0.4 → 0.8s) is the industry-standard way to retry without hammering a struggling server; it's in every serious HTTP client, message queue and cloud SDK.

বাস্তব দৃশ্য: রাতের জব একটি খামখেয়ালি পেমেন্ট API ডাকে যা মাঝে মাঝে টাইমআউট হয়। প্রয়োজন: ৩ বার রিট্রাই, প্রতিবার দ্বিগুণ অপেক্ষা (এক্সপোনেনশিয়াল ব্যাকঅফ), প্রতি চেষ্টার লগ, আর ফাইল কখনো পুরোটা মেমোরিতে নয়। `retry(times=3)` একটি ডেকোরেটর **ফ্যান্টরি** — ডেকোরেটর ফেরত দেওয়া ফাংশন, তাই তিন স্তর। ব্যাকঅফ (0.2 → 0.4 → 0.8s) ধুঁকতে থাকা সার্ভারকে না পিটিয়ে রিট্রাই করার শিল্প-মান — প্রতিটি ক্লাউড SDK-তে এটাই আছে।

6.7 Common Mistakes

Mistake	Symptom	Fix
Looping a generator twice	Second loop gets nothing	Generators are one-shot; recreate or <code>list()</code> once
<code>print(gen)</code> expecting values	<code><generator object ...></code>	Iterate it, or <code>list(gen)</code> for debugging
Decorator without <code>*args, **kwargs</code>	Breaks on functions with arguments	Wrapper signature is always <code>(*args, **kwargs)</code>
Missing <code>@functools.wraps</code>	Wrong <code>__name__</code> in logs; broken tooling	Add it to every wrapper
<code>groupby</code> on unsorted data	Duplicate, fragmented groups	Sort by the same key first
<code>lru_cache</code> on functions with side effects	Emails sent once, "cached" forever	Cache pure computations only

ভুলসমূহ: জেনারেটর দ্বিতীয়বার লুপ (এক-বার-ব্যবহার্য — খালি ফেরত); `print(gen)`-এ মান আশা করা; ডেকোরেটরের র‍্যাপারে `*args, **kwargs` না দেওয়া; `@functools.wraps` ভোলা; না-সাজানো ডেটায় `groupby`; আর সাইড-ইফেক্টওয়ালা ফাংশনে `lru_cache` (ইমেইল একবার যাবে, বাকি সময় "ক্যাশড")।

6.8 Practice Exercises (with Answers)

- `batched(iterable, n)` **generator**: yield lists of `n` items at a time — the pattern for bulk-inserting rows into a database 500 at a time.
- `@require_role("admin")` **decorator factory**: wrapped function takes a `user` dict; raise `PermissionError` unless `user["role"]` matches.
- Add full type hints to the lab's `retry`, `charge` and `stream_orders`.

PYTHON

```
# --- Answer 1 ---
def batched(iterable, n):
    batch = []
    for item in iterable:
        batch.append(item)
        if len(batch) == n:
            yield batch; batch = []
    if batch:
        yield batch # don't drop the final partial batch

print(list(batched(range(7), 3))) # [[0,1,2], [3,4,5], [6]]

# --- Answer 2 ---
import functools
def require_role(role):
    def deco(func):
        @functools.wraps(func)
        def wrapper(user, *args, **kwargs):
            if user.get("role") != role:
                raise PermissionError(f"{role} only")
            return func(user, *args, **kwargs)
        return wrapper
```

```
return wrapper
return deco
```

বাংলা

অনুশীলন: (১) `batched` জেনারেটর — একসাথে n -টি করে দেয়; ডাটাবেসে ৫০০ সারি করে বাল্ক-ইনসার্টের প্যাটার্ন (শেষ অসম্পূর্ণ ব্যাচটি ফেলবেন না!); (২) `@require_role("admin")` ডেকোরেটর ফ্যাক্টরি — ভুল রোলে `PermissionError`; (৩) ল্যাবের কোডে সম্পূর্ণ টাইপ হিন্ট। উত্তর দুটির গঠন ৬.১ ও ৬.২-এর হুবহু প্রয়োগ।

Production Python: Real Tools for Real Jobs

প্রোডাকশন পাইথন: বাস্তব কাজের হাতিয়ার

IN THIS CHAPTER · এই অধ্যায়ে

Logging instead of print · argparse CLIs · calling web APIs safely · SQLite with injection-proof queries · pytest · threads vs asyncio · Capstone: currency fetcher → database → daily report.

7.1 Logging Done Right (Stop Using print)

Plain-language first: `print` talks to a human watching the screen; **logging** writes a permanent, timestamped, filterable record — the flight recorder of your program. When a nightly job fails at 3 AM, the log file is the only witness.

PYTHON

```
import logging

logging.basicConfig(
    level=logging.INFO,
    format="%(asctime)s %(levelname)-8s %(name)s: %(message)s",
    handlers=[logging.FileHandler("app.log", encoding="utf-8"),
              logging.StreamHandler()],          # file AND console
)
log = logging.getLogger("billing")

log.debug("raw payload: %s", data)      # diagnostic detail (hidden at INFO level)
log.info("invoice %s generated", inv_id)
log.warning("stock below reorder point")
log.error("payment gateway rejected txn")
try:
    risky()
except Exception:
    log.exception("charge failed")      # error + full traceback, automatically
```

Levels let you keep detailed `debug` calls in the code forever and simply raise the threshold in production.

`log.exception(...)` inside `except` captures the whole traceback — the single most useful logging call.

বাংলা

`print` কথা বলে স্ক্রিনের সামনে বসা মানুষের সাথে; **লগিং** লেখে স্থায়ী, টাইমস্ট্যাম্পসহ, ফিল্টারযোগ্য রেকর্ড — প্রোগ্রামের ব্ল্যাকবক্স। রাত ৩টায় জব ফেল করলে লগ ফাইলই একমাত্র সাক্ষী। লেভেল (debug < info < warning < error) দিয়ে কোডে বিস্তারিত `debug` রেখে দিয়ে প্রোডাকশনে শুধু থ্রেশহোল্ড বাড়ান। `except` ব্লকে `log.exception(...)` — এরর + পুরো ট্রেসব্যাক স্বয়ংক্রিয়ভাবে — লগিং-এর সবচেয়ে দরকারি কল।

7.2 Command-Line Tools with argparse

Real scripts take options instead of editing code or answering `input()` prompts — that's what makes them schedulable by cron/Task Scheduler and usable by teammates:

PYTHON

```
# report.py
import argparse

parser = argparse.ArgumentParser(description="Generate the daily sales report")
parser.add_argument("date", help="report date, YYYY-MM-DD")
parser.add_argument("--branch", default="all", help="branch code")
parser.add_argument("--top", type=int, default=3, help="how many top products")
```



```
parser.add_argument("--json", action="store_true", help="emit JSON not text")
args = parser.parse_args()

print(args.date, args.branch, args.top, args.json)
```

TERMINAL

```
python report.py 2026-08-19 --branch DHK --top 5
python report.py --help          # free, auto-generated documentation
```

`type=int` converts and *validates* for you; `action="store_true"` makes a flag; `--help` comes free. The moment a script has two knobs, give it argparse.

বাংলা

বাস্তব স্ক্রিপ্ট কোড এডিট বা `input()` নয় — কমান্ড-লাইন অপশন নেয়; তবেই cron-এ শিডিউল করা যায়, সহকর্মী ব্যবহার করতে পারে। `argparse`-এ পজিশনাল আর্গুমেন্ট, `--branch`-এর মতো অপশন, `type=int`-এ স্বয়ংক্রিয় রূপান্তর+যাচাই, `action="store_true"`-তে ফ্ল্যাগ — আর `--help` ডকুমেন্টেশন ফ্রি। স্ক্রিপ্ট দুটি নব থাকলেই argparse দিন।

7.3 Calling Web APIs with requests

An **API** (Application Programming Interface) is a URL that returns data (JSON) instead of a web page. The third-party `requests` library (`pip install requests`) is the standard client:

PYTHON

```
import requests

resp = requests.get(
    "https://api.frankfurter.dev/v1/latest",
    params={"base": "USD", "symbols": "BDT,EUR"}, # -> ?base=USD&symbols=...
    timeout=10,                                # NEVER call without a timeout
)
resp.raise_for_status() # raises for 4xx/5xx instead of silent bad data
data = resp.json()      # parsed straight into a dict
print(data["rates"]["BDT"])

resp = requests.post("https://httpbin.org/post",
                    json={"sku": "R-77", "qty": 2}, timeout=10)
```

The professional skeleton is always the same four lines: request with `timeout` → `raise_for_status()` → `.json()` → use the dict with chapter-3 skills. Status codes: 200 OK · 201 created · 400 your request is wrong · 401/403 auth problem · 404 not found · 429 slow down · 500 their server broke (retry with backoff — 6.6!).

⚠ WARNING

API keys are passwords. Never hard-code them: read from an environment variable — `os.environ["API_KEY"]` — and keep them out of git.

বাংলা

API হলো এমন URL যা ওয়েবপেজ নয়, ডেটা (JSON) ফেরত দেয়; `requests` লাইব্রেরি তার স্ট্যান্ডার্ড ক্লায়েন্ট। পেশাদার কক্ষাল সবসময় এক: `timeout` সহ রিকোয়েস্ট → `raise_for_status()` (8xx/5xx-এ নীরব খারাপ ডেটার বদলে এরর) → `.json()` → ডিক্ট নিয়ে কাজ। স্ট্যাটাস কোড চিনুন: 200 ঠিক আছে, 400 আপনার ভুল, 401/403 অনুমতি নেই, 429 ধীরে, 500 ওদের সার্ভার ভাঙা (ব্যাকঅফসহ রিট্রাই)। **API key পাসওয়ার্ড** — কোডে নয়, environment variable-এ রাখুন।

7.4 SQLite: A Real Database with Zero Setup

SQLite is a full SQL database living in a single file — no server to install, built into Python, and powering phones, browsers and countless products. Perfect for tools, prototypes and anything below heavy multi-user load:

PYTHON

```
import sqlite3

con = sqlite3.connect("shop.db")
con.execute("""CREATE TABLE IF NOT EXISTS rates(
    day TEXT PRIMARY KEY,
    usd_bdt REAL NOT NULL)""")

with con:
    con.execute("INSERT OR REPLACE INTO rates VALUES (?, ?)",
        ("2026-08-19", 118.42))    # ? placeholders – ALWAYS

for day, rate in con.execute(
    "SELECT day, usd_bdt FROM rates ORDER BY day DESC LIMIT 7"):
    print(day, rate)
con.close()
```

`with con:` wraps statements in a transaction — all succeed or none apply. And the `?` placeholders are non-negotiable: building SQL with f-strings invites **SQL injection**, the attack where user input like `' ; DROP TABLE rates; --` becomes part of your query.

বাংলা

SQLite হলো এক ফাইলের পূর্ণাঙ্গ SQL ডাটাবেস — সার্ভার লাগে না, Python-এ বিল্ট-ইন; ফোন-ব্রাউজার সবখানে এটাই চলে। `with con:` ব্লক ট্রানজ্যাকশন — সব সফল নয়তো কিছুই না। **সবচেয়ে জরুরি নিয়ম:** কোয়েরিতে মান বসান `?` প্লেসহোল্ডারে, কখনো f-string-এ নয় — নইলে **SQL ইনজেকশন:** ব্যবহারকারীর ইনপুট (`' ; DROP TABLE ...--`) আপনার কোয়েরির অংশ হয়ে যায়।

7.5 Testing with pytest

Tests are code that verifies your code — the safety net that lets you change things without fear. `pip install pytest`, put functions named `test_*` in a file named `test_*.py`, assert plainly:

PYTHON

```
# billing.py
def slab_bill(units: int) -> float:
    if units < 0:
        raise ValueError("units cannot be negative")
    energy = (min(units, 75) * 5.26
        + min(max(units - 75, 0), 125) * 7.20
        + max(units - 200, 0) * 9.94)
    return round((energy + 42) * 1.05, 2)

# test_billing.py
import pytest
from billing import slab_bill

def test_zero_units_pays_only_demand_charge():
    assert slab_bill(0) == 44.1

def test_slab_boundary_75():
    assert slab_bill(75) == round((75 * 5.26 + 42) * 1.05, 2)

def test_negative_units_rejected():
    with pytest.raises(ValueError):
        slab_bill(-5)
```

TERMINAL

```
pytest -q          # ... 3 passed in 0.02s
```

What to test first: **boundaries** (0, exactly 75, exactly 200) and **error paths** — that's where real bugs live. Notice the payoff of chapter-2 discipline: because `slab_bill` *returns* instead of printing and touches no globals, testing it is trivial.

টেস্ট হলো কোড যাচাইকারী কোড — এই নিরাপত্তা-জালের কারণেই ভয় ছাড়া কোড বদলানো যায়। `test_*.py` ফাইলে `test_*` ফাংশন, ভেতরে সরল `assert`; `pytest -q` সব চালায়। আগে কী টেস্ট করবেন: **সীমানা** (০, ঠিক ৭৫, ঠিক ২০০) আর **এরর-পথ** — বাগ ওখানেই থাকে। খেয়াল করুন: ফাংশন `print` না করে **return** করে আর গ্লোবাল ছোঁয় না বলেই টেস্ট করা এত সহজ — দ্বিতীয় অধ্যায়ের শৃঙ্খলার প্রতিদান।

7.6 Concurrency in One Page: Threads & asyncio

Downloading 20 URLs one-by-one wastes time *waiting* on the network. For waiting-heavy (I/O-bound) work, run the waits in parallel:

PYTHON

```
import requests
from concurrent.futures import ThreadPoolExecutor

urls = [f"https://httpbin.org/delay/1?i={i}" for i in range(10)]

def fetch(url: str) -> int:
    return requests.get(url, timeout=15).status_code

with ThreadPoolExecutor(max_workers=10) as pool:
    results = list(pool.map(fetch, urls))    # ~1s total instead of ~10s
```

Decision table you'll use for years:

Workload	Nature	Tool
API calls, downloads, DB waits	I/O-bound	<code>ThreadPoolExecutor</code> (simple) or <code>asyncio</code> (thousands of tasks)
Heavy math, image/video processing	CPU-bound	<code>multiprocessing</code> / <code>ProcessPoolExecutor</code>
A dozen-ish network calls	I/O-bound, small	<code>ThreadPoolExecutor</code> — don't over-engineer

`asyncio` (`async def` / `await`) scales to tens of thousands of concurrent connections and powers modern frameworks like FastAPI — meet it after threads feel comfortable. Threads don't speed up pure Python math (the Global Interpreter Lock, GIL, allows one thread to run Python at a time); processes do.

২০টি URL একে একে নামানো মানে বেশিরভাগ সময় নেটওয়ার্কের **অপেক্ষায়** নষ্ট। অপেক্ষা-প্রধান (I/O-bound) কাজে `ThreadPoolExecutor` দিয়ে অপেক্ষাগুলো সমান্তরালে চালান — ১০ সেকেন্ডের কাজ ~১ সেকেন্ডে। সিদ্ধান্ত-সূত্র: নেটওয়ার্ক/ডিস্ক অপেক্ষা → থ্রেড (বা বিশাল স্কেলে `asyncio`); ভারী গণনা → `multiprocessing` (GIL-এর কারণে থ্রেডে খাঁটি Python অঙ্ক দ্রুত হয় না)। অল্প কয়েকটা কলে জটিলতা বাড়াবেন না — থ্রেডপুলই যথেষ্ট।

7.7 Capstone Lab: Currency Fetcher → SQLite → Daily Report

Real scenario: finance wants today's USD → BDT rate pulled from a public API every morning, stored with history, and a 7-day report printed — resilient enough for cron. One file, every chapter on duty:

PYTHON

```
# rate_tracker.py — capstone: API + DB + logging + argparse + error handling
import argparse, logging, sqlite3, sys
from datetime import date
import requests

log = logging.getLogger("rates")

def get_rate(base: str = "USD", target: str = "BDT") -> float:
    resp = requests.get("https://api.frankfurter.dev/v1/latest",
                        params={"base": base, "symbols": target}, timeout=10)
    resp.raise_for_status()
```

```

return float(resp.json()["rates"][target])

def save_rate(con: sqlite3.Connection, day: str, rate: float) -> None:
    with con:
        con.execute("INSERT OR REPLACE INTO rates VALUES (?, ?)", (day, rate))

def weekly_report(con: sqlite3.Connection) -> str:
    rows = list(con.execute(
        "SELECT day, usd_bdt FROM rates ORDER BY day DESC LIMIT 7"))
    lines = [f"{d} {r:8.2f}" for d, r in rows]
    if len(rows) >= 2:
        change = rows[0][1] - rows[-1][1]
        lines.append(f"7-day change: {change:+.2f}")
    return "\n".join(lines)

def main() -> int:
    p = argparse.ArgumentParser(description="Track daily USD/BDT rate")
    p.add_argument("--db", default="rates.db")
    p.add_argument("--report-only", action="store_true")
    args = p.parse_args()

    logging.basicConfig(level=logging.INFO,
        format="%(asctime)s %(levelname)s %(message)s",
        handlers=[logging.FileHandler("rates.log"), logging.StreamHandler()])

    con = sqlite3.connect(args.db)
    con.execute("""CREATE TABLE IF NOT EXISTS rates(
        day TEXT PRIMARY KEY, usd_bdt REAL NOT NULL)""")
    try:
        if not args.report_only:
            rate = get_rate()
            save_rate(con, date.today().isoformat(), rate)
            log.info("saved %.2f", rate)
        print(weekly_report(con))
        return 0 # exit code 0 = success (cron reads this)
    except requests.RequestException:
        log.exception("fetch failed - keeping yesterday's data")
        print(weekly_report(con)) # degrade gracefully: still report history
        return 1
    finally:
        con.close()

if __name__ == "__main__":
    sys.exit(main())

```

TERMINAL

```

python rate_tracker.py # fetch + store + report
python rate_tracker.py --report-only # offline: report from history

```

This is the anatomy of a production script: small typed functions each doing one job; the network call isolated (so tests can fake it); transactions on writes; logs to file *and* console; **graceful degradation** — if the API is down it still reports history and returns exit code 1 so cron can alert; `finally` guarantees the connection closes. Extend it: `--base EUR` option, a `ThreadPoolExecutor` fetching 5 currency pairs at once (7.6), a pytest file for `weekly_report` (7.5).

বাংলা

ক্যাপস্টোন দৃশ্য: প্রতি সকালে API থেকে USD → BDT রেট এনে ইতিহাসসহ ডাটাবেসে রাখা, ৭ দিনের রিপোর্ট ছাপা — cron-এ চলার মতো মজবুত। প্রোডাকশন স্ক্রিপ্টের শারীরস্থান দেখুন: ছোট ছোট টাইপ-হিটসহ ফাংশন, নেটওয়ার্ক কল আলাদা (টেস্টে নকল করা যাবে), লেখায় ট্রানজ্যাকশন, ফাইল+কনসোল লগ, আর **গ্রেসফুল ডিগ্রেডেশন** — API পড়ে গেলেও ইতিহাস থেকে রিপোর্ট দেয়, exit code 1 ফেরত দেয় যাতে cron অ্যালার্ট করতে পারে। নিজে বাড়ান: `--base EUR` অপশন, ৫টি মুদ্রা থ্রেডপুলে একসাথে, `weekly_report`-এর pytest।

7.8 Production Checklist

Before any script touches real data or a schedule, walk this list:

- Every external call (network, file, DB) has a **timeout** and specific `except` handling; failures are **logged with `log.exception`**, never swallowed.
- Secrets come from **environment variables**, never the source code or git history.
- SQL uses `?` placeholders; user-facing input is validated with guard clauses (4.3).
- The script **returns exit codes** (0 success / non-zero failure) so schedulers can alert.
- Config lives in arguments or a JSON/env file — no editing code to change behaviour.
- Core logic is in pure, returning functions with **type hints** and at least boundary **tests**.
- Big data streams through **generators**; repeated pure lookups sit behind `lru_cache`.
- A `README` line says how to install (`pip install -r requirements.txt`) and run it.

বাংলা

প্রোডাকশন চেকলিস্ট: প্রতিটি বাহ্যিক কলে timeout + নির্দিষ্ট except + `log.exception`; সিক্রেট environment variable-এ; SQL-এ `?` প্লেসহোল্ডার; exit code ফেরত (শিডিউলার অ্যালার্ট করবে); কনফিগ আর্গুমেন্ট/ফাইলে; মূল লজিক টাইপ-হিন্টসহ বিশুদ্ধ ফাংশনে + সীমানা-টেস্ট; বড় ডেটা জেনারেটরে; ইনস্টল-রান নির্দেশনা README-তে। এই তালিকা পাস করলে স্ক্রিপ্ট সত্যিই শিপ করার যোগ্য।

Cheat Sheet, Error Decoder & Next Steps

চিটশিট, এরর ডিকোডার ও পরবর্তী ধাপ

IN THIS CHAPTER · এই অধ্যায়ে

One-page syntax cheat sheet · a decoder table for the 11 errors you will actually meet · ten graded real projects · the roadmap after this book.

A.1 One-Page Cheat Sheet

PYTHON

```
# ---- types & conversion ----
int("42") float("3.5") str(99) round(3.456, 2) type(x)
# ---- strings ----
s.strip() s.lower() s.split(",") ", ".join(lst) s.replace(a,b)
s.startswith(p) "x" in s s[2:5] f"{v:,.2f}" f"{v:>10}"
# ---- list / dict / set ----
lst.append(x) lst.pop() sorted(lst, key=lambda t: t[1], reverse=True)
d[k] d.get(k, default) d.setdefault(k, []).append(v) d.items()
d[k] = d.get(k, 0) + 1 # counting pattern
set(lst) a & b a - b a | b # dedupe & set algebra
[f(x) for x in xs if cond] {k: v for k, v in pairs}
# ---- flow & functions ----
for i, x in enumerate(xs, 1): ... # numbered
for a, b in zip(xs, ys): ... # parallel
def f(x: int, rate: float = 0.05) -> float: return ...
# ---- files / json / csv ----
with open(p, encoding="utf-8") as f: [line for line in f]
json.dump(obj, f, indent=2, ensure_ascii=False); obj = json.load(f)
csv.DictReader(f) csv.DictWriter(f, fieldnames=[...])
# ---- errors ----
try: ...
except (ValueError, KeyError) as e: log.exception(...)
raise ValueError("why") # guard clause
# ---- advanced ----
def gen(): yield item # streaming
@functools.wraps(func) # inside every decorator
@lru_cache(maxsize=256) # memoise pure functions
with sqlite3.connect(db) as con: con.execute("... VALUES (?,?)", vals)
requests.get(url, params=p, timeout=10).raise_for_status()
```

বাংলা

এক পাতার চিটশিট — বইয়ের প্রতিটি মূল অঙ্ক এক জায়গায়: টাইপ রূপান্তর, সাত স্ট্রিং মেথড, গণনা ও গ্রুপিং প্যাটার্ন, কমপ্রিহেনশন, `enumerate` / `zip`, ফাইল-JSON-CSV, গার্ড ক্লজ, জেনারেটর, ডেকোরেটর, ক্যাশ, SQLite ও requests-এর নিরাপদ কল্লাল। প্রিন্ট করে ডেস্কে রাখুন।

A.2 Error Message Decoder

Error	Usual meaning	First thing to check
<code>SyntaxError: expected ':'</code>	Missing colon on <code>if</code> / <code>for</code> / <code>def</code> line	End of the reported line
<code>IndentationError</code>	Wrong/mixed indentation	4 spaces, no tabs
<code>NameError: name 'x' is not defined</code>	Typo, or used before assignment	Spelling; order of lines
<code>TypeError: can only concatenate str...</code>	Mixing str and int	<code>int()</code> your <code>input()</code> / CSV values
<code>ValueError: invalid literal for int()</code>	Converting non-numeric text	Print the raw value first
<code>KeyError: 'price'</code>	Dict key missing	<code>d.get("price")</code> ; print <code>d.keys()</code>
<code>IndexError: list index out of range</code>	Position past the end	<code>len()</code> vs the index; empty list?
<code>AttributeError: 'NoneType' object...</code>	A function returned <code>None</code>	The call before — <code>sort()</code> ? missing <code>return</code> ?
<code>FileNotFoundError</code>	Wrong path/working directory	<code>import os; print(os.getcwd())</code>
<code>ModuleNotFoundError</code>	Package not in <i>this</i> venv	Is the venv activated? <code>pip install</code> again
<code>UnicodeDecodeError</code>	Encoding mismatch	Add <code>encoding="utf-8"</code>

Read tracebacks bottom-up: last line names the error, the line above it points at your code.

বাংলা

এরর ডিকোডার: `SyntaxError` = কোলন/বানান; `NameError` = টাইপো বা আগে-ব্যবহার; `TypeError` str+int মেশানো; `KeyError` ডিক্টে কী নেই; `AttributeError: 'NoneType'` মানে আগের কোনো কল `None` দিয়েছে (`sort()` ? `return` নেই?); `ModuleNotFoundError` = venv সক্রিয় নেই। ট্রেসব্যাক **নিচ থেকে উপরে** পড়ুন — শেষ লাইনে এররের নাম, তার উপরে আপনার কোডের জায়গা।

A.3 Ten Real Projects to Build Next

Ordered by difficulty; each names the chapters it exercises:

1. **Expense splitter CLI** — who owes whom after a group trip (1–3).
2. **File organiser** — sort a messy Downloads folder into dated subfolders by extension (4, `pathlib`).
3. **Password vault (local)** — JSON storage, master-PIN gate, logout from 2.6 (2, 4).
4. **Website uptime monitor** — ping your sites every 5 min, log status, alert on change (6, 7).
5. **CSV bank-statement analyzer** — monthly totals by category, top merchants (3, 4).
6. **Attendance system** — SQLite backed, argparse commands `checkin` / `report` (5, 7).
7. **Weather CLI** — public API, cache responses 10 min with a timestamp check (7).
8. **Bulk image resizer** — `Pillow` + `ThreadPoolExecutor` over a folder (6, 7.6).
9. **Price-drop tracker** — scheduled fetch, SQLite history, "notify if below X" (full 7).
10. **Mini REST API** — expose your inventory lab over HTTP with FastAPI (everything).

Rule: build without looking at the book first; open it only when stuck 20+ minutes.

বাংলা

পরবর্তী দশ প্রজেক্ট (সহজ → কঠিন): খরচ ভাগকারী, ফাইল সংগঠক, লোকাল পাসওয়ার্ড ভল্ট, আপটাইম মনিটর, ব্যাংক-স্টেটমেন্ট বিশ্লেষক, SQLite হাজিরা সিস্টেম, ক্যাশসহ আবহাওয়া CLI, বাল্ক ইমেজ রিসাইজার, প্রাইস-ড্রপ ট্র্যাকার, FastAPI দিয়ে মিনি REST API। নিয়ম: আগে বই না দেখে বানান; ২০ মিনিট আটকে থাকলে তবেই খুলুন।

A.4 Where to Go Next

- **Official docs & tutorial** — docs.python.org: the reference you'll use forever.
- **Web**: FastAPI (modern, typed) or Django (batteries included).
- **Data**: pandas → matplotlib → then ML with scikit-learn if that's your path.
- **Automation/DevOps**: `pathlib`, `subprocess`, `schedule`, then Docker basics.
- **Craft**: run `ruff` (linter/formatter) and `mypy` on everything; read *Fluent Python* when chapters 5–6 feel easy.
- Keep the loop from 0.5: build → break → read the traceback → fix. That loop, repeated for months, is the entire secret.

বাংলা

এরপরের পথ: অফিসিয়াল ডকুমেন্টেশন সারাজীবনের রেফারেন্স; ওয়েবে FastAPI বা Django; ডেটায় pandas → matplotlib → scikit-learn; অটোমেশনে pathlib/subprocess/schedule ও Docker; মান বাড়াতে সব কোডে `ruff` ও `mypy` চালান। আর ০.৫-এর চক্রটাই আসল রহস্য: বানান → ভাঙুন → ট্রেসব্যাক পড়ুন → ঠিক করুন — মাসের পর মাস।